Memoria del Proyecto Final

Introducción a Python

Xabier Sánchez

Índice

1. Ordenación temporal, latencia y relojes distribuidos **3**
1.1. Básicas . . . 3
1.2. Avanzadas . . . 4

2. Persistencia, idempotencia y eliminación de duplicados **9**
2.1. Básicas . . . 9
2.2. Avanzadas . . . 10

3. Backpressure y control de flujo **13**
3.1. Básicas . . . 13
3.2. Avanzadas . . . 14

4. Consistencia eventual vs consistencia fuerte en PnL **20**
4.1. Básicas . . . 20
4.2. Avanzadas . . . 21

5. Modelado avanzado de posiciones y agregados **25**
5.1. Básicas . . . 25
5.2. Avanzadas . . . 26

6. Concurrencia, asincronismo y paralelismo en Python **30**
6.1. Básicas . . . 30
6.2. Avanzadas . . . 32

7. Uso avanzado de Python: decoradores, metaclases y dunder methods **38**
7.1. Básicas . . . 38
7.2. Avanzadas . . . 40

8. Escalado horizontal y multiactivo **45**
8.1. Básicas . . . 45
8.2. Avanzadas . . . 47

9. Tolerancia a fallos y recuperación determinista **51**
9.1. Básicas . . . 51
9.2. Avanzadas . . . 52

10. Pruebas avanzadas y validación del sistema **57**
10.1. Básicas . . . 57

11. Proceso de diseño, retos encontrados y decisiones de simplificación **59**
11.1. Proceso de diseño del sistema . . . 59
11.2. Justificación de las principales elecciones . . . 60
11.3. Retos encontrados y cómo se resolvieron . . . 62
11.4. Decisiones de simplificación adoptadas . . . 63

1. Ordenación temporal, latencia y relojes distribuidos

1.1. Básicas

- Explica cómo has detectado que los WebSockets de Binance pueden llegar a entregar mensajes fuera de orden. Pon un ejemplo concreto.
- ¿Qué diferencia hay entre event time e ingestion time? ¿Cuál usarías para cálculos y por qué?
- ¿Cómo garantizas en Python que una secuencia de timestamps cumpla monoticidad temporal? Explica cómo lo validarías.

1.1.1. Ordenación temporal, event time e ingestion time

Para detectar posibles mensajes fuera de orden en el WebSocket implementé un Monotonicity-Validator. Este componente mantiene, para cada símbolo, el mayor event_time recibido hasta el momento. Cada vez que llega un nuevo trade, compara su timestamp con ese máximo. Si el nuevo valor es inferior, significa que el evento ha llegado después que otro que, según el reloj del exchange, ocurrió más tarde, por lo que se registra una violación de monotonicidad.

La comprobación se realiza de forma independiente para cada activo, ya que la secuencia temporal de BTCUSDT no debe compararse con la de ETHUSDT. Además, utilizo una comparación estricta <y no <=, porque varios trades pueden compartir el mismo timestamp en milisegundos sin que exista desorden. El validador tampoco reduce el máximo cuando recibe un evento atrasado: conserva siempre el mayor timestamp observado, de modo que puede detectar varios eventos retrasados consecutivos.

Un ejemplo concreto utilizado en los tests es la siguiente secuencia:

```
BTCUSDT: 1000 → 1005 → 1005 → 1002 → 1010
```

El segundo valor 1005 es válido porque la monotonicidad exigida es no decreciente. Sin embargo, cuando llega 1002, el sistema ya había observado 1005, por lo que detecta un retroceso temporal de:

```
1005 − 1002 = 3 ms
```

El test comprueba que se registra exactamente una violación para BTCUSDT y que su magnitud es de 3 ms. También existen tests que verifican que los símbolos se controlan de forma independiente y que el máximo temporal no disminuye al recibir datos atrasados.

En una ejecución real del WebSocket procesé aproximadamente 5.900 trades y obtuve violaciones=0. Por tanto, no puedo afirmar que en esa muestra concreta Binance entregase mensajes fuera de orden. Lo que sí puedo demostrar es que el pipeline comprueba cada trade recibido y está preparado para detectar y cuantificar el problema cuando ocurra. El consumidor ejecuta la validación antes de persistir cada evento y aumenta el contador de violaciones cuando el timestamp retrocede.

El **event time** es el instante en el que el trade fue ejecutado en Binance. Este valor lo proporciona el exchange en el propio mensaje y representa el tiempo económico del evento. El **ingestion time** es el instante en el que mi aplicación recibió ese mensaje desde el WebSocket, medido con el reloj del ordenador donde se ejecuta el ingestor. En el código, el ingestion time se registra inmediatamente después de recibir el mensaje y antes de introducirlo en la cola, para no mezclar el retraso de red con el posible retraso interno del consumidor.

Para ordenar trades, construir velas y realizar cálculos financieros utilizaría el event_time, porque los cálculos deben respetar el orden en el que las operaciones ocurrieron en el mercado, no el orden accidental en el que fueron recibidas por mi proceso. El ingestion_time lo utilizaría principalmente para observabilidad: medir retrasos, detectar problemas de red, analizar el lag del sistema y estudiar el comportamiento de la cola.

La diferencia entre ambos tiempos puede expresarse como:

```
latencia aparente = ingestion_time − event_time
```

Durante la ejecución real obtuve una media aproximada de −1502 ms. Una latencia de red real no puede ser negativa, por lo que este resultado indica que el reloj local estaba aproximadamente 1,5 segundos atrasado respecto al reloj utilizado por Binance. Esto demuestra que la resta entre timestamps de máquinas distintas también contiene el desfase entre relojes. Por tanto, antes de interpretar esta diferencia como latencia real es necesario sincronizar los relojes o estimar y corregir su offset.

La validación se implementa en el método check() de MonotonicityValidator. El objeto mantiene un diccionario, _last_seen_ms, con el mayor timestamp observado para cada símbolo. Al recibir un nuevo evento, recupera el valor anterior mediante self._last_seen_ms.get(symbol) y comprueba si el nuevo event_time_ms es estrictamente inferior.

Si existe un retroceso, crea un objeto MonotonicityViolation, lo añade a la lista de violaciones y no modifica el máximo previamente observado. Si el timestamp avanza o se repite, actualiza el diccionario con el nuevo valor.

La validación se comprueba mediante tests deterministas con secuencias creadas manualmente, porque de esta forma conozco de antemano el número exacto de violaciones esperado. Esto hace que el test no dependa de la red, de Binance ni de que se produzca casualmente un evento fuera de orden durante su ejecución.

1.2. Avanzadas

- Describe cómo reconstruyes un flujo temporal coherente cuando llegan ticks fuera de orden. Explica ventajas e inconvenientes de ordenar por buffer vs. aplicar corrección incremental.
- Define clock drift. ¿En qué casos el timestamp del backend es autoritativo sobre el del exchange?

1.2.1. Reconstrucción de un flujo cuando llegan ticks fuera de orden

Mi implementación actual detecta los retrocesos temporales, pero no reordena automáticamente los eventos. MonotonicityValidator conserva el mayor event_time observado por símbolo y registra una violación cuando llega un timestamp inferior. Además, no reduce el máximo tras recibir un evento atrasado, por lo que varios ticks tardíos consecutivos siguen siendo detectados.

Para reconstruir un flujo temporal coherente utilizaría un **buffer acotado por símbolo**. Cada tick recibido se introduciría en una estructura ordenada por event_time, por ejemplo una cola de prioridad. El sistema retendría los eventos durante una ventana temporal pequeña, como 500 ms o 1 s, antes de publicarlos al consumidor.

El algoritmo sería:

1. Recibir el tick y asignarle su ingestion_time.
2. Insertarlo en el buffer correspondiente a su símbolo, ordenado por event_time.

3. Mantener el mayor event_time observado para ese símbolo.
4. Calcular una marca de agua o *watermark*:

```
watermark = máximo event_time observado − ventana de tolerancia
```

5. Emitir, en orden, todos los ticks cuyo event_time sea menor o igual que la marca de agua.
6. Si posteriormente llega un evento anterior a la marca de agua, tratarlo como un evento tardío y aplicar una política explícita: registrarlo, corregir resultados afectados o enviarlo a una cola separada.

Por ejemplo, si se reciben los eventos:

```
1000 → 1005 → 1002 → 1010
```

y existe una ventana de tolerancia suficiente, el evento 1002 permanece temporalmente en el buffer y la salida puede reconstruirse como:

```
1000 → 1002 → 1005 → 1010
```

El buffer debe ser acotado tanto por tiempo como por número de elementos. De lo contrario, un retraso prolongado o la caída de un consumidor podría provocar un crecimiento ilimitado de memoria.

1.2.2. Ordenación por buffer frente a corrección incremental

La **ordenación mediante buffer** retrasa temporalmente el procesamiento para esperar posibles eventos tardíos.

Sus ventajas son:

- Los consumidores reciben una secuencia temporal coherente.
- Simplifica los cálculos posteriores, porque velas, indicadores y posiciones reciben datos ya ordenados.
- Evita tener que deshacer resultados calculados previamente.
- Su comportamiento puede ser determinista si se define una ventana y una política de marca de agua.

Sus inconvenientes son:

- Añade latencia artificial: para reordenar es necesario esperar.
- No garantiza orden perfecto si un tick llega después de cerrar la ventana.
- Consume memoria.
- Exige elegir una ventana de tolerancia. Una ventana muy pequeña deja pasar desorden; una demasiado grande perjudica la frescura.

La **corrección incremental** procesa los eventos inmediatamente, aunque puedan llegar desordenados. Si posteriormente llega un tick tardío, se localizan y recalculan únicamente los resultados afectados.

Por ejemplo, si un trade tardío pertenece a una vela de las 14:00, el sistema puede:

1. Insertar el trade tardío de forma idempotente.
2. Reconstruir la vela de las 14:00.
3. Volver a calcular los indicadores y señales posteriores que dependan de ella.
4. Actualizar las métricas o el PnL afectados.

Sus ventajas son:

- La latencia de publicación es menor.
- No obliga a detener el flujo esperando eventos que quizá nunca lleguen.
- Es adecuada cuando la mayoría de los eventos llegan ordenados y los retrasos son excepcionales.

Sus inconvenientes son:

- La implementación es bastante más compleja.
- Los resultados ya publicados pueden cambiar.
- Es necesario conocer qué cálculos dependen del evento corregido.
- Puede producir cascadas de recálculo, especialmente en indicadores con ventanas temporales largas.
- Requiere mecanismos de versionado, auditoría o notificación para evitar que dos consumidores observen estados incompatibles.

Para este proyecto elegiría un **buffer pequeño por símbolo para el flujo en tiempo real** y persistiría también los eventos tardíos. Si un evento superase la ventana del buffer, aplicaría una corrección incremental sobre las velas y métricas afectadas. Esta solución híbrida evita introducir una latencia excesiva y, al mismo tiempo, permite corregir retrasos anómalos.

La clave primaria (symbol, trade_id) garantiza que una corrección o una reconexión no introduzcan dos veces el mismo trade.

1.2.3. Clock drift

El **clock drift** es la variación progresiva de un reloj respecto a una referencia temporal correcta. Los osciladores físicos de dos equipos no avanzan exactamente a la misma velocidad, por lo que, aunque ambos relojes se sincronicen inicialmente, con el paso del tiempo pueden volver a separarse.

Conviene distinguirlo del **clock offset** o desfase:

- El offset es la diferencia entre dos relojes en un instante concreto.
- El drift es la velocidad a la que esa diferencia cambia con el tiempo.

En la ejecución real del ingestor observé una latencia media aproximada de −1502 ms. El cálculo utilizado era:

```
ingestion_time − event_time
```

El resultado negativo indica que el reloj local estaba aproximadamente 1,5 segundos atrasado respecto al de Binance. Esta observación evidencia un offset; para demostrar drift sería necesario repetir la medición durante un periodo más largo y comprobar si ese desfase aumenta o disminuye progresivamente.

El event_time del trade se obtiene de Binance, mientras que el ingestion_time se estampa localmente justo después de recibir el mensaje y antes de introducirlo en la cola. Esto permite separar la latencia externa del retraso interno de procesamiento.

1.2.4. ¿Cuándo es autoritativo el timestamp del backend?

Para determinar cuándo ocurrió económicamente un trade, el timestamp del exchange es el autoritativo. Binance es el sistema que ejecutó la operación y conoce el instante en que esta entró en su secuencia de mercado. Por ello, utilizaría el event_time para:

- ordenar trades de un mismo símbolo;
- construir velas;
- calcular indicadores;
- generar señales;
- realizar backtesting;
- reconstruir la historia del mercado.

El timestamp del backend es autoritativo para hechos que ocurren dentro de mi propio sistema. Por ejemplo:

- cuándo recibí el mensaje;
- cuándo entró en la cola;
- cuándo fue persistido;
- cuánto tiempo permaneció esperando;
- cuándo respondió la API;
- si se cumplió un SLA interno;
- en qué orden observó los mensajes mi aplicación.

También podría utilizarse como referencia provisional si el exchange no proporciona timestamp, si el campo llega corrupto o si se está registrando un evento generado por el propio backend. Sin embargo, en esos casos no debería presentarse como tiempo económico del mercado, sino como tiempo de observación local.

Por tanto, no existe un reloj universalmente autoritativo para todo. La autoridad depende del hecho que se quiere medir:

```
Ejecución en el mercado        → event_time del exchange
Recepción y procesamiento local → timestamp del backend
```

2. Persistencia, idempotencia y eliminación de duplicados

2.1. Básicas

- Explica cómo identificas duplicados tras una reconexión al WebSocket. ¿Cuál es la primary identity de un trade?
- Define idempotencia en el contexto de tu ingestor de datos. Da un caso donde una implementación ingenua generaría duplicados.
- Describe tu mecanismo (independiente del tipo de almacenamiento) para evitar registrar dos veces el mismo trade bajo duplicados y reintentos.

2.1.1. Identificación de duplicados y primary identity

Tras una reconexión al WebSocket, el exchange puede reenviar trades que mi aplicación ya había procesado. Para identificar esos duplicados no comparo todos los campos del evento, sino su identidad de mercado.

La **primary identity** de un trade es:

(symbol, trade_id)

El trade_id es único dentro de cada símbolo, pero no necesariamente entre símbolos diferentes. Por tanto, el trade 12345 de BTCUSDT y el trade 12345 de ETHUSDT son eventos distintos. Esta misma identidad se utiliza tanto en las entidades de Python como en la clave primaria de SQLite. La clase Trade redefine ___eq___ y ___hash___ para que dos objetos con el mismo (symbol, trade_id) se consideren el mismo trade, aunque otros campos, como el ingestion_time, sean distintos. Esto puede ocurrir cuando un mismo evento se recibe dos veces en momentos diferentes.

2.1.2. Idempotencia en el ingestor

En este contexto, una operación es **idempotente** cuando procesar varias veces el mismo trade produce el mismo estado final que procesarlo una sola vez.

Por ejemplo, si tras una reconexión se recibe dos veces:

BTCUSDT, trade_id=12345

el resultado correcto es que exista una sola fila almacenada, no dos.

Una implementación ingenua podría limitarse a ejecutar un INSERT o añadir cada evento a un archivo según llega. Si el WebSocket se desconecta después de recibir un trade pero antes de confirmar hasta dónde había procesado, al reconectarse podría volver a recibirlo y almacenarlo una segunda vez. Esto alteraría el volumen negociado, el número de trades y cualquier métrica calculada a partir de ellos.

2.1.3. Mecanismo para evitar duplicados

El principio general, independiente del almacenamiento, es mantener una clave de deduplicación estable para cada evento y hacer que el registro sea atómico:

```
clave de deduplicación = (symbol, trade_id)
```

Antes de aceptar un evento, el sistema debe comprobar si esa clave ya fue procesada. Si ya existe, el evento se considera duplicado y no modifica el estado. Si no existe, se registra la clave junto con el evento como una única operación lógica.

En mi implementación con SQLite, esta garantía se delega en la base de datos mediante:

```
PRIMARY KEY (symbol, trade_id)
```

y:

```
INSERT OR IGNORE
```

La base de datos decide atómicamente si el trade es nuevo. Si la clave ya existe, ignora la segunda inserción. Esto evita el problema del patrón ingenuo:

```
comprobar si existe → insertar
```

porque entre ambas operaciones otro proceso podría insertar el mismo trade y provocar una condición de carrera.

El método insert_trade() devuelve:

```
True  → el trade fue insertado
False → ya existía y fue ignorado
```

El consumidor utiliza ese resultado para incrementar por separado los contadores de trades insertados y duplicados.

El comportamiento también está verificado mediante un test determinista: se fabrica un evento de Binance, se transforma en una entidad Trade y se intenta insertar dos veces. La primera inserción devuelve True, la segunda devuelve False y el número final de filas permanece en uno.

Por tanto, el sistema ofrece una semántica práctica de procesamiento **at least once con efectos idempotentes**: un evento puede recibirse más de una vez, pero su efecto persistido se aplica una sola vez.

2.2. Avanzadas

- Si tu almacenamiento no soporta transacciones, explica tu solución para aplicar idempotencia (por ejemplo índices en memoria, atomic rename, offsets persistidos).
- Explica cómo evitas condiciones de carrera al insertar datos desde varios hilos o procesos, indicando el mecanismo de sincronización elegido.
- Si usas SQL, detalla tu esquema mínimo para garantizar unicidad bajo concurrencia.

2.2.1. Idempotencia sin transacciones

Si utilizase un almacenamiento sin transacciones, mantendría un índice de claves ya procesadas:

```
clave = (symbol, trade_id)
```

Un índice únicamente en memoria no sería suficiente, porque se perdería al reiniciar el proceso. Por ello, combinaría tres elementos:

1. Un log append-only con los trades recibidos.
2. Un índice persistido con las claves procesadas.
3. Un offset o posición que indique hasta qué punto del log se ha confirmado el procesamiento.

Las actualizaciones del índice y del offset se escribirían primero en un archivo temporal y después se sustituiría el archivo anterior mediante atomic rename. De esta forma, ante una caída, se conservaría la versión antigua completa o la nueva, pero no un archivo parcialmente escrito.

Al reiniciar, el sistema cargaría el último índice válido y reproduciría el log desde el offset confirmado. Si encuentra una clave ya registrada, ignora el evento; si no existe, procesa el trade y actualiza el índice. Esta solución reproduce la propiedad idempotente aunque el almacenamiento no proporcione transacciones.

El índice en memoria serviría como caché rápida, pero la garantía real dependería del log y del índice persistido.

2.2.2. Condiciones de carrera entre hilos o procesos

Una condición de carrera aparecería si dos escritores ejecutasen una lógica de este tipo:

1. Comprobar si el trade existe.

2. Si no existe, insertarlo.

Ambos podrían comprobar simultáneamente que no existe y después insertar el mismo evento.

En mi implementación actual evito esta situación mediante dos niveles de sincronización.

En el nivel de aplicación utilizo un único consumidor de la asyncio.Queue. El productor solo recibe y encola eventos, mientras que el consumidor los valida y persiste secuencialmente utilizando una única conexión. Por tanto, dentro del ingestor no existen varios escritores compitiendo entre sí.

En el nivel de almacenamiento no realizo un SELECT previo. Ejecuto directamente:

```
INSERT OR IGNORE INTO trades (...)
```

La comprobación de unicidad y la inserción se resuelven como una operación atómica dentro de SQLite. Si dos procesos intentasen insertar simultáneamente el mismo (symbol, trade_id), solo uno podría crear la fila y el otro sería ignorado por la restricción de clave primaria.

SQLite se configura además en modo WAL. Esto permite que el backend y el dashboard sigan leyendo mientras el ingestor escribe. SQLite serializa las escrituras, por lo que puede haber un solo escritor efectivo en cada instante, aunque existan varias conexiones.

Si el número de escritores creciera, mantendría una arquitectura de **single writer**: varios productores enviarían eventos a una cola común y un único proceso se encargaría de escribir por lotes. Esto reduce la contención y evita errores de bloqueo. La implementación actual no configura busy_timeout, por lo que, con muchos procesos escritores, sería necesario añadirlo o migrar a una base de datos preparada para mayor concurrencia, como PostgreSQL.

2.2.3. Esquema SQL mínimo para garantizar unicidad

El esquema mínimo de la tabla de trades es:

```
CREATE TABLE trades (
    symbol             TEXT    NOT NULL,
    trade_id           INTEGER NOT NULL,
    price              REAL    NOT NULL,
    qty                REAL    NOT NULL,
    event_time_ms      INTEGER NOT NULL,
    ingestion_time_ms  INTEGER NOT NULL,
    is_buyer_maker     INTEGER NOT NULL,
    PRIMARY KEY (symbol, trade_id)
```

) WITHOUT ROWID;

La clave primaria compuesta es la garantía principal:

```
PRIMARY KEY (symbol, trade_id)
```

trade_id es único dentro de cada símbolo, pero el mismo identificador puede existir para activos distintos. Por eso no sería correcto utilizar únicamente trade_id.

WITHOUT ROWID hace que la clave primaria compuesta sea la estructura principal de la tabla, evitando mantener un identificador interno y un índice adicional.

La operación de inserción es:

```
INSERT OR IGNORE INTO trades
    (symbol, trade_id, price, qty, event_time_ms,
     ingestion_time_ms, is_buyer_maker)
VALUES (?, ?, ?, ?, ?, ?, ?);
```

La restricción se comprueba dentro de la base de datos y bajo su control de concurrencia. Esto garantiza que dos inserciones concurrentes con la misma identidad no puedan producir dos filas. Una se confirma y la otra se ignora.

Para las velas utilizo el mismo principio, pero con otra identidad:

```
PRIMARY KEY (symbol, interval, open_time_ms)
```

Así se evita duplicar una vela al repetir una descarga histórica.

3. Backpressure y control de flujo

3.1. Básicas

- Define qué es backpressure y dónde aparece en tu sistema.
- Explica qué ocurre si Binance envía datos más rápido de lo que procesas.
- ¿Cómo detectas en Python que tu consumidor va con retraso (lag)?

3.1.1. ¿Qué es backpressure y dónde aparece?

El **backpressure** es el mecanismo mediante el cual un consumidor lento obliga al productor a reducir su ritmo, evitando que los datos pendientes crezcan indefinidamente en memoria.

En mi sistema aparece entre el productor del WebSocket y el consumidor que persiste los trades:

```
Binance WebSocket → productor → asyncio.Queue → consumidor → SQLite
```

La cola se crea con un tamaño máximo:

```
queue = asyncio.Queue(maxsize=1000)
```

Cuando el productor recibe un trade ejecuta:

```
await queue.put(trade)
```

Si la cola alcanza los 1.000 elementos, queue.put() no añade más datos inmediatamente, sino que suspende al productor hasta que el consumidor libera espacio. Esta espera cooperativa constituye el mecanismo de backpressure del pipeline.

3.1.2. ¿Qué ocurre si Binance envía datos más rápido de lo que se procesan?

Si Binance produce eventos más rápido de lo que el consumidor puede validarlos y almacenarlos, los trades empiezan a acumularse en la cola.

Mientras quede espacio, el sistema absorbe temporalmente la diferencia de velocidades:

```
velocidad de entrada > velocidad de consumo
```

→ aumenta el número de elementos en la cola

Si la cola llega a su capacidad máxima, el productor queda suspendido en await queue.put(). Con esta política no se descartan deliberadamente trades, por lo que se prioriza la integridad de los datos frente a la frescura.

El inconveniente es que, al bloquearse el productor, también deja de leer temporalmente del socket. Si el atasco dura demasiado, pueden llenarse los buffers de red o Binance puede cerrar la conexión por lentitud. El ingestor intentaría reconectarse mediante backoff exponencial. Para recuperar cualquier posible hueco sería necesario realizar posteriormente un backfill mediante REST; esta recuperación está planteada en el diseño, pero no se ejecuta automáticamente en la implementación actual.

Una alternativa sería descartar los eventos más antiguos, conocida como política drop oldest. Esto mantendría el flujo actualizado, pero introduciría huecos silenciosos y sesgaría métricas

como volumen, número de operaciones o construcción de velas. Por ello, en este proyecto he elegido bloquear al productor antes que perder datos.

3.1.3. ¿Cómo detecto que el consumidor va con retraso?

La métrica básica utilizada es el tamaño actual de la cola:

```
queue.qsize()
```

El consumidor lo muestra cada 100 trades junto con los insertados, duplicados, violaciones de monotonicidad y latencia media.

La interpretación es:

- Cola cercana a cero y estable: el consumidor procesa los datos al ritmo de llegada.
- Picos que después vuelven a cero: existen ráfagas, pero el consumidor consigue recuperarse.
- Cola que crece continuamente: el consumidor acumula retraso.
- Cola próxima a maxsize durante un periodo prolongado: existe riesgo de bloqueo del productor y desconexión.

En la ejecución real observé valores como:

```
16 → 7 → 24 → 4 → 0 → 1
```

Aunque hubo pequeñas ráfagas, la cola volvió repetidamente a cero. Por tanto, durante esa muestra el consumidor fue capaz de alcanzar al productor y no apareció un lag sostenido.

Para una instrumentación más completa añadiría también la edad del trade más antiguo de la cola:

```
lag temporal = tiempo actual − ingestion_time del evento más antiguo pendiente
```

qsize() indica cuántos eventos están esperando, mientras que esta segunda métrica mostraría cuánto tiempo lleva esperando el evento más atrasado.

3.2. Avanzadas

- Compara bounded queues con drop oldest y rate limiting. ¿Qué sesgos introduce cada una en tus métricas?
- Define métricas de lag y diseña una alerta basada en instrumentación real.
- Describe cómo reequilibrarías el throughput del pipeline bajo carga extrema.

3.2.1. Bounded queues, drop oldest y rate limiting

En mi implementación utilizo una **cola acotada**:

```
queue = asyncio.Queue(maxsize=1000)
```

Cuando la cola se llena, await queue.put(trade) suspende al productor hasta que el consumidor libera espacio. No se pierden eventos deliberadamente, pero aumenta la latencia y, si el bloqueo se prolonga, el proceso puede dejar de leer el WebSocket con suficiente rapidez.

Las principales alternativas son:

3.2.2. Bounded queue con bloqueo

- Conserva todos los eventos recibidos.
- Limita el uso de memoria.
- Transmite la presión del consumidor hacia el productor.
- Puede incrementar el lag y provocar una desconexión si el bloqueo dura demasiado.

Esta política no sesga directamente las métricas de volumen o número de trades, porque no elimina observaciones. Sin embargo, sí puede reducir la frescura del dashboard: los datos son completos, pero llegan tarde.

3.2.3. Drop oldest

Cuando la cola está llena, se elimina el evento más antiguo para admitir uno nuevo.

- Mantiene el sistema próximo al tiempo real.
- Evita que el productor quede bloqueado.
- Introduce pérdida explícita de información.

Esta política sesga especialmente:

- volumen negociado;
- número de trades;
- precios medios ponderados;
- construcción de velas;
- detección de máximos y mínimos;
- métricas de latencia.

Además, al eliminar los eventos más antiguos de la cola, la latencia aparente de los eventos procesados disminuye artificialmente. El sistema parece más rápido porque ha descartado precisamente los mensajes que más tiempo llevaban esperando.

3.2.4. Rate limiting

El productor limita voluntariamente la frecuencia de entrada o procesamiento.

Esta solución es útil cuando el sistema controla el origen de las peticiones, como ocurre en el cliente REST, donde utilizo un semáforo para limitar las descargas concurrentes.

Sin embargo, en un WebSocket no puedo obligar a Binance a generar menos trades. Solo podría limitar operaciones posteriores, como actualizaciones del dashboard o cálculos de estrategias.

Si aplicase muestreo, agregación o procesamiento de uno de cada varios ticks, introduciría sesgos similares a la pérdida de datos:

- infravaloración del volumen;
- menor número de eventos;
- posible pérdida de movimientos extremos;
- señales construidas sobre una muestra incompleta.

Por tanto, para datos financieros elegiría una cola acotada con bloqueo, combinada con persistencia eficiente y un mecanismo de recuperación mediante REST si se pierde la conexión.

3.2.5. Métricas de lag

El tamaño de la cola es la métrica más inmediata:

```
queue.qsize()
```

El consumidor ya la muestra cada 100 trades. No obstante, qsize() por sí sola no es suficiente: una cola de 100 eventos puede representar unos pocos milisegundos en un mercado activo o varios segundos en uno poco líquido.

Utilizaría las siguientes métricas:

```
queue_depth = número de eventos pendientes
queue_utilization = queue_depth / queue_maxsize
```

queue_lag_ms =

```
tiempo actual − ingestion_time del evento más antiguo pendiente
```

processing_lag_ms =

```
tiempo de persistencia − ingestion_time
```

throughput_in =

eventos recibidos por segundo

throughput_out =

eventos persistidos por segundo

También mediría:

- número de veces que la cola llega a su capacidad máxima;
- duración acumulada del productor bloqueado;
- duplicados y violaciones de monotonicidad;
- tiempo de escritura en SQLite;
- percentiles p50, p95 y p99 del tiempo de procesamiento.

La diferencia entre entrada y salida permite anticipar el atasco:

```
throughput_in > throughput_out
```

→ la cola crecerá

La latencia calculada actualmente como ingestion_time − event_time no debe utilizarse directamente como métrica de lag interno, porque mezcla la latencia de red con el desfase entre el reloj de Binance y el reloj local. En la ejecución real produjo aproximadamente −1502 ms, evidenciando ese desfase.

3.2.6. Diseño de una alerta

Definiría dos niveles:

Warning:

```
queue_utilization > 50 % durante 10 segundos
o queue_lag_ms > 1.000 ms
```

Critical:

```
queue_utilization > 80 % durante 30 segundos
o queue_lag_ms > 5.000 ms
o throughput_out < throughput_in de forma sostenida
```

No alertaría por un pico aislado, porque durante la ejecución real la cola subió hasta 24 elementos y después volvió a cero. Exigir que la condición se mantenga durante varios segundos evita falsos positivos causados por ráfagas normales.

También generaría una alerta inmediata si:

```
queue_depth == maxsize
```

porque indica que el productor está siendo bloqueado y existe riesgo de desconexión.

3.2.7. Reequilibrio del throughput bajo carga extrema

El principal cuello de botella actual es la persistencia. sqlite3 es síncrono y insert_trade() realiza un commit por cada trade. Aunque esto es suficiente para el volumen observado, bajo miles de eventos por segundo bloquearía el event loop y reduciría considerablemente el throughput. Aplicaría las mejoras en este orden:

3.2.8. 1. Inserciones por lotes

El consumidor acumularía, por ejemplo, 100 o 500 trades y los insertaría dentro de una única transacción:

```
100 trades → 1 commit
```

en lugar de:

```
100 trades → 100 commits
```

Esto reduciría el coste de sincronización con disco y aumentaría notablemente el throughput.

El lote se enviaría cuando alcanzase un tamaño máximo o transcurriese un tiempo breve, por ejemplo 100 ms, para no introducir una latencia excesiva cuando el mercado esté tranquilo.

3.2.9. 2. Sacar SQLite del event loop

La escritura bloqueante podría ejecutarse mediante:

```
await asyncio.to_thread(write_batch, batch)
```

Así, el event loop seguiría recibiendo mensajes mientras otro hilo realiza la operación de disco.

Mantendría un único escritor para evitar contención entre varias conexiones SQLite.

3.2.10. 3. Separar las etapas del pipeline

Usaría varias colas acotadas:

WebSocket

```
    ↓
```

cola de recepción

```
    ↓
```

validación y normalización

```
    ↓
```

cola de persistencia

```
    ↓
```

escritor por lotes

De este modo podría medir el lag de cada etapa y localizar el cuello de botella exacto.

3.2.11. 4. Particionar por símbolo

La validación y los cálculos podrían repartirse por símbolo:

```
worker 1 → BTCUSDT, ETHUSDT
worker 2 → XRPUSDT, BNBUSDT
worker 3 → resto de símbolos
```

Cada worker mantendría su propio estado de monotonicidad, evitando compartir y bloquear un diccionario global.

La escritura final seguiría centralizada para SQLite. Si se migrase a PostgreSQL, sería posible utilizar varios escritores concurrentes manteniendo la restricción única sobre (symbol, trade_id).

3.2.12. 5. Separar trabajo de I/O y CPU

La recepción WebSocket y la persistencia son tareas principalmente I/O-bound, adecuadas para asyncio y threads. El cálculo intensivo de estrategias o reconstrucción de velas podría enviarse a procesos separados para evitar la limitación del GIL.

3.2.13. 6. Particionar las conexiones WebSocket

Con diez símbolos una conexión combinada es suficiente. Con cientos de activos repartiría los símbolos entre varias conexiones y procesos. Esto evitaría que el atasco o la caída de una sola conexión afectase a todo el universo de activos.

En conjunto, mantendría el principio de **integridad antes que pérdida silenciosa**: primero optimizaría la persistencia, el batching y la paralelización. Solo aplicaría descarte o muestreo en elementos no contables, como actualizaciones visuales del dashboard, pero no en los trades utilizados para construir velas, posiciones o PnL.

4. Consistencia eventual vs consistencia fuerte en PnL

4.1. Básicas

- Indica qué partes necesitan consistencia fuerte para calcular correctamente el PnL.
- Da un ejemplo donde un retraso o inconsistencia genera un PnL incorrecto.

4.1.1. Partes que necesitan consistencia fuerte

Para calcular correctamente el PnL necesito que los datos que intervienen pertenezcan a un estado coherente del sistema. En concreto, requieren consistencia fuerte:

- Las velas utilizadas para calcular los retornos.
- Las señales generadas por la estrategia.
- La posición efectiva aplicada en cada periodo.
- Los pesos de los activos del portafolio.
- El histórico usado para calcular el PnL diario, mensual, QTD, YTD y total.

En mi implementación, el PnL se calcula siempre a partir de velas ya persistidas en SQLite y no directamente desde el estado en memoria del WebSocket. Esto permite que dos cálculos sobre el mismo conjunto de datos produzcan el mismo resultado y que el estado sobreviva a un reinicio.

La posición utilizada en cada vela se obtiene desplazando la señal una posición:

```
positions = signals.shift(1).fillna(0)
```

Esto es necesario porque la señal de la vela t solo se conoce cuando dicha vela ha cerrado. Por tanto, no puede aplicarse al retorno de esa misma vela, sino a partir de la siguiente.

También es necesario que todos los activos del portafolio se calculen sobre un corte temporal compatible. Para una consistencia estricta entre varios símbolos utilizaría una única transacción de lectura o un identificador de snapshot, de forma que todas las consultas vean el mismo estado de la base de datos.

Por el contrario, algunos elementos del sistema pueden ser eventualmente consistentes sin afectar al cálculo contable, como:

- la tabla visual de últimos trades;
- el gráfico del dashboard;
- el último precio mostrado;
- las métricas de latencia;
- el refresco de la interfaz.

El dashboard puede tardar unos segundos en reflejar los cambios, pero el PnL no debería mezclar datos de instantes distintos.

4.1.2. Ejemplo de PnL incorrecto por retraso o inconsistencia

Supongamos un portafolio equiponderado formado por BTC y ETH. En una determinada hora:

BTC: +10 %

ETH: −10 %

El retorno correcto del portafolio sería:

```
(+10 % − 10 %) / 2 = 0 %
```

Si la última vela de BTC ya está almacenada, pero la de ETH todavía no ha llegado o no se ha persistido, el cálculo podría utilizar únicamente el retorno de BTC para ese timestamp y mostrar temporalmente:

```
PnL del portafolio = +10 %
```

En lugar del resultado correcto de 0 %.

Este riesgo es relevante en mi implementación porque pandas alinea las series por timestamp y calcula la media ignorando los valores ausentes. Cuando un activo no tiene información para una fecha, el portafolio se calcula con los activos disponibles y sus pesos se redistribuyen implícitamente.

Por ello, antes de calcular el PnL debería comprobar que todos los activos esperados tienen cerrada y persistida la misma vela, o definir explícitamente una política para los datos ausentes. Algunas opciones serían esperar a que estén disponibles todos los símbolos, utilizar únicamente el último timestamp común o marcar el resultado como provisional.

Otro ejemplo sería calcular el retorno de una vela utilizando una señal generada al cierre de esa misma vela. Sin el desplazamiento temporal, el sistema capturaría un movimiento que todavía no podía conocerse, produciendo un PnL artificialmente elevado. Este error está cubierto por un test donde una señal aparece justo en la vela de un salto del 100 % y se comprueba que no puede capturarlo.

4.2. Avanzadas

- Describe condiciones de carrera en cálculos de PnL y cómo las has eliminado.
- Explica qué partes del sistema pueden ser eventualmente consistentes sin afectar decisiones.
- Demuestra cómo validarías invariantes de PnL en tests unitarios.

4.2.1. Condiciones de carrera en el cálculo del PnL

Una condición de carrera aparecería si el PnL se calculase al mismo tiempo que el ingestor está escribiendo nuevas velas y cada activo se leyese en un instante diferente. Por ejemplo, BTC podría cargarse después de persistir su última vela y ETH justo antes de persistir la suya. El resultado mezclaría dos estados temporales distintos del mercado.

En mi diseño reduzco este riesgo calculando el PnL exclusivamente desde datos persistidos en SQLite, nunca desde estructuras temporales del WebSocket. El endpoint abre una conexión por petición, carga las velas almacenadas, genera las señales y calcula el portafolio a partir de ese estado.

SQLite utiliza modo WAL, lo que permite que el ingestor escriba mientras el backend realiza lecturas sin que ambos accedan a estructuras de memoria compartidas. La base de datos mantiene el aislamiento de cada consulta frente a escrituras incompletas.

También elimino una condición de carrera temporal dentro del propio backtest mediante:

```
positions = signals.shift(1).fillna(0)
```

La señal de la vela t solo puede convertirse en posición en la vela siguiente. Sin este desplazamiento, el cálculo utilizaría información conocida al cierre de una vela para capturar el retorno de esa misma vela, introduciendo *look-ahead bias*.

No obstante, mi implementación actual realiza una consulta independiente por símbolo. Por ello, no garantiza de forma estricta que los diez activos se lean desde un único snapshot temporal. Para eliminar completamente ese riesgo utilizaría una transacción de lectura común y calcularía el PnL únicamente hasta el último timestamp cerrado disponible para todos los activos.

4.2.2. Partes que pueden ser eventualmente consistentes

La consistencia eventual es aceptable en los elementos visuales o informativos que no intervienen en una decisión financiera inmediata. En mi sistema pueden actualizarse con cierto retraso:

- El candlestick mostrado en el dashboard.
- La tabla de últimos trades.
- Los marcadores visuales de señales.
- El último precio mostrado.
- Las métricas de latencia y observabilidad.
- La lista de símbolos y estrategias.

El dashboard utiliza un refresco por polling cada dos segundos y cachés con distintos TTL. Los datos vivos, como velas, PnL y trades, tienen un TTL de dos segundos; los datos casi estáticos, como símbolos y estrategias, tienen un TTL de cinco minutos.

Que el gráfico tarde dos segundos en reflejar un trade no modifica el dato persistido ni el cálculo posterior. En cambio, la contabilidad del PnL, las posiciones y las señales utilizadas para tomar decisiones deben basarse en datos coherentes y reproducibles.

En resumen:

Consistencia fuerte:

velas persistidas, señales, posiciones, pesos y PnL.

Consistencia eventual:

dashboard, últimos trades, métricas y elementos decorativos.

4.2.3. Validación de invariantes de PnL

Un invariante es una propiedad que debe cumplirse para cualquier conjunto válido de datos, no solo para un ejemplo concreto. En el proyecto valido varios invariantes mediante tests unitarios deterministas.

4.2.4. 1. Sin posición no puede existir PnL

Si todas las señales son cero, la posición efectiva también debe ser cero y ningún movimiento del mercado puede generar beneficios o pérdidas:

```
candles = make_candles([100, 150, 80, 200, 50])
signals = pd.Series(0, index=candles.index)

returns = strategy_returns(candles, signals)

assert (returns == 0).all()
```

Este test evita el denominado PnL fantasma: dinero generado sin haber mantenido una posición.

4.2.5. 2. Los retornos deben componerse, no sumarse

Para un activo que sube dos periodos consecutivos un 10 %:

```
100 → 110 → 121
```

el retorno total correcto es:

```
1,10 × 1,10 − 1 = 21 %
```

El test comprueba que el PnL acumulado utiliza composición geométrica y no una suma incorrecta del 20 %.

4.2.6. 3. Una señal no puede capturar el movimiento que la genera

Con cierres:

```
100 → 100 → 200
```

y señales:

```
0 → 0 → 1
```

la señal aparece al cierre de la vela en la que el precio ya ha subido un 100 %. Al aplicar shift(1), la posición efectiva sigue siendo cero durante ese salto, por lo que el PnL debe ser exactamente cero.

```
returns = strategy_returns(candles, signals)
assert (returns == 0).all()
```

El mismo test incluye el contraste: si la señal aparece una vela antes, entonces sí debe capturar el movimiento completo. Este es el principal test contra *look-ahead bias*.

4.2.7. 4. El corto debe ser el espejo del largo

Con retornos simples, aplicar una señal -1 debe producir, vela a vela, el opuesto de aplicar una señal $+1$:

```
assert ((long_returns + short_returns).abs() < 1e-12).all()
```

Esto comprueba que la dirección de la posición se aplica correctamente.

4.2.8. 5. Una serie vacía debe devolver PnL cero

Si todavía no existen datos para un activo o estrategia, el resumen no debe lanzar una excepción ni inventar resultados:

```
assert pnl_summary(pd.Series(dtype=float)) == {
    "daily": 0.0,
    "monthly": 0.0,
    "qtd": 0.0,
    "ytd": 0.0,
    "total": 0.0,
}
```

Estos tests protegen clases completas de errores: PnL sin posición, composición incorrecta, anticipación temporal, signos invertidos y fallos en casos límite.

5. Modelado avanzado de posiciones y agregados

5.1. Básicas

- Define la entidad responsable de mantener la posición agregada. Explica su invariante.
- Explica con un ejemplo cómo actualizas el precio medio tras varios partial fills.

La entidad responsable de mantener la posición agregada es Position. Cada instancia representa el estado de un único símbolo mediante:

```
qty          → cantidad abierta con signo
avg_price    → precio medio de la cantidad que sigue abierta
realized_pnl → beneficio o pérdida ya realizado
```

Una qty positiva representa una posición larga y una negativa, una posición corta. La clase es inmutable: al aplicar un nuevo Fill, el método ___add___ devuelve una nueva posición en lugar de modificar la anterior.

El invariante principal es que avg_price debe ser siempre el precio medio ponderado por volumen de la cantidad que permanece abierta. Además:

- aumentar la posición recalcula el precio medio;
- reducirla no cambia el precio medio de lo que queda abierto;
- el PnL realizado solo cambia al reducir o cerrar posición;
- si la posición queda plana, qty y avg_price pasan a cero;
- el coste base de la posición abierta es qty × avg_price.

Por ejemplo, compro 10 unidades a 100 y después otras 10 a 120. Como ambas operaciones aumentan una posición larga, el nuevo precio medio es:

```
avg_price = (10 × 100 + 10 × 120) / 20 = 110
```

La posición queda:

```
qty = 20
avg_price = 110
realized_pnl = 0
```

Si después vendo parcialmente 5 unidades a 130, la cantidad abierta baja a 15, pero el precio medio de las unidades restantes continúa siendo 110. La venta realiza:

```
realized_pnl = 5 × (130 − 110) = 100
```

El estado final es:

```
qty = 15
avg_price = 110
realized_pnl = 100
```

La regla fundamental es:

```
Aumentar posición → recalcular el precio medio.
Reducir posición  → mantener el precio medio y realizar PnL.
```

Este comportamiento está verificado con tests unitarios que cubren compras sucesivas, ventas parciales, cierres completos, posiciones cortas y cruces por cero.

5.2. Avanzadas

- ¿Puedes reconstruir la posición exacta si falta un tramo del histórico? ¿Qué datos son imprescindibles?
- Diseña un algoritmo determinista para reconstruir el portafolio solo a partir del log de trades.
- Explica problemas si tus entidades mezclan conceptos específicos de Binance con el dominio.

5.2.1. ¿Puede reconstruirse la posición exacta si falta un tramo del histórico?

No siempre. La posición actual es el resultado acumulado de todas las ejecuciones anteriores, por lo que, si falta un tramo del log de Fill, normalmente no puede reconstruirse de forma exacta.

Por ejemplo, conocer únicamente que la posición final es larga no permite saber:

- cuántas compras y ventas parciales se realizaron;
- qué precio medio quedó abierto;
- cuánto PnL se realizó durante el tramo perdido;
- si la posición cruzó por cero y cambió de larga a corta;
- qué cantidad permanecía abierta al comenzar el periodo disponible.

Para reconstruirla exactamente necesito una de estas dos opciones:

1. El historial completo y ordenado de fills desde una posición inicial conocida, normalmente una posición plana.
2. Un snapshot fiable anterior al tramo disponible, que incluya qty, avg_price y realized_pnl, más todos los fills posteriores.

Si falta un tramo y tampoco existe un snapshot anterior, pueden existir varias historias compatibles con los datos disponibles, pero con distintos precios medios y PnL realizados. Por tanto, el resultado no sería identificable de forma única.

Los datos imprescindibles de cada ejecución propia son:

symbol

cantidad con signo

precio de ejecución

orden temporal o secuencia de ejecución

identificador único del fill

La cantidad debe tener signo: positiva para una compra y negativa para una venta. El orden es fundamental porque aplicar fills en distinto orden puede modificar el PnL realizado y el momento en que se produce un cruce por cero.

En la implementación actual, Fill contiene únicamente symbol, qty y price. Esto es suficiente para los tests deterministas, donde el orden viene dado por la propia lista, pero para un sistema persistido real añadiría al menos un fill_id y un timestamp o número de secuencia.

5.2.2. Reconstrucción determinista desde el log de fills

La reconstrucción comienza con una posición plana para cada símbolo:

```
Position(symbol, qty=0, avg_price=0, realized_pnl=0)
```

Después se ordenan los fills mediante una clave estable, por ejemplo:

(event_time, fill_id)

El fill_id sirve como desempate cuando dos ejecuciones comparten timestamp.

A continuación, se aplican secuencialmente:

```
positions[symbol] = positions[symbol] + fill
```

El método Position.___add___() contempla tres situaciones:

1. **Aumento de posición:** recalcula el precio medio ponderado.
2. **Reducción o cierre:** conserva el precio medio y materializa PnL.
3. **Flip:** cierra la posición anterior y abre el sobrante en sentido contrario al precio del nuevo fill.

Conceptualmente, el algoritmo sería:

```
def rebuild_portfolio(fills):
    positions = {}

    ordered_fills = sorted(
        fills,
        key=lambda fill: (fill.event_time_ms, fill.fill_id),
    )

    for fill in ordered_fills:
        if fill.symbol not in positions:
            positions[fill.symbol] = Position(fill.symbol)

        positions[fill.symbol] = positions[fill.symbol] + fill

    return positions
```

El proyecto ya implementa esta idea para un símbolo mediante position_from_fills(), que aplica un reduce sobre la lista de fills. Dado el mismo estado inicial y el mismo log en el mismo orden, el resultado siempre es idéntico.

También existe un test que reconstruye dos veces una posición desde el mismo log y comprueba que ambos resultados son iguales campo a campo.

Es importante distinguir entre **trades de mercado** y **fills propios**. La entidad Trade del proyecto representa operaciones públicas ejecutadas en Binance, mientras que Fill representa una ejecución de mi propia cartera. No podría reconstruir mi posición únicamente a partir del flujo público de trades de Binance, porque ese flujo no indica cuáles de esas operaciones pertenecen a mi cuenta.

5.2.3. Problemas de mezclar Binance con el dominio

El dominio debería expresar conceptos financieros generales, no el formato particular de un proveedor.

Binance envía campos con nombres como:

s, t, p, q, T, m

Si esas claves se propagasen por todo el sistema, las estrategias, el almacenamiento y el cálculo de posiciones dependerían directamente de Binance. Cambiar de exchange obligaría a modificar muchas capas.

Por eso, parse_trade() actúa como frontera: traduce el mensaje específico de Binance a una entidad de dominio con nombres como:

symbol

trade_id

price

qty

event_time_ms

is_buyer_maker

Las claves específicas del exchange no salen del módulo de ingesta.

Mezclar ambos niveles produciría varios problemas:

- Acoplamiento fuerte a Binance.
- Dificultad para incorporar otro exchange.
- Tests dependientes del formato externo.
- Entidades con campos sin significado fuera de Binance.
- Confusión entre operaciones públicas de mercado y ejecuciones propias.
- Propagación de cambios de API por todo el proyecto.

En mi diseño, las entidades de dominio solo dependen de la biblioteca estándar y no importan httpx, websockets ni sqlite3. Esto permite probarlas de manera aislada y sustituir Binance por otro proveedor cambiando principalmente el adaptador de ingesta.

Por tanto, mantengo la separación:

```
Formato de Binance → adaptador de ingesta → entidad de dominio
```

La infraestructura conoce al proveedor; el dominio solo conoce conceptos como trade, fill y posición.

6. Concurrencia, asincronismo y paralelismo en Python

6.1. Básicas

- Explica cuándo usar asyncio y cuándo usar threads.
- Define qué problema resuelve el GIL. ¿Cómo afecta al procesamiento en tu sistema?
- ¿Cómo asegurarías que una función bloqueante no congela tu evento loop?

6.1.1. ¿Cuándo utilizar asyncio y cuándo utilizar threads?

Utilizaría asyncio principalmente para tareas **I/O-bound** que ofrecen una interfaz asíncrona, es decir, operaciones que pasan gran parte del tiempo esperando a la red, a una cola o a otro recurso externo.

En mi proyecto se utiliza en:

- la recepción de mensajes del WebSocket;
- las peticiones REST mediante httpx.AsyncClient;
- la espera entre reconexiones;
- el control de concurrencia mediante asyncio.Semaphore;
- la comunicación entre productor y consumidor mediante asyncio.Queue.

Mientras una corrutina espera una respuesta de red o un elemento de la cola, el event loop puede ejecutar otra tarea. Por ejemplo, el productor puede esperar mensajes de Binance mientras el consumidor procesa los trades que ya están encolados. Ambas corrutinas se ejecutan concurrentemente dentro de un solo hilo. Utilizaría **threads** cuando necesito integrar una función bloqueante que no dispone de una versión asíncrona, especialmente si se trata de I/O. Algunos ejemplos serían:

- una librería de base de datos síncrona;
- lectura o escritura de archivos;
- una API externa que solo ofrece funciones bloqueantes;
- llamadas a código legado.

En mi implementación, sqlite3 es síncrono. Actualmente el consumidor llama directamente a insert_trade(), algo aceptable para el volumen observado, pero que puede bloquear el event loop si la base de datos tarda demasiado.

No utilizaría threads como primera opción para cálculos intensivos de CPU. Para ese tipo de trabajo preferiría procesos separados mediante multiprocessing o ProcessPoolExecutor, porque los procesos permiten ejecutar bytecode de Python realmente en paralelo.

En resumen:

```
I/O asíncrono disponible       → asyncio
I/O bloqueante sin API async   → threads
Cálculo intensivo de CPU       → procesos
```

6.1.2. El GIL y su efecto en el sistema

El **GIL**, o *Global Interpreter Lock*, es un bloqueo utilizado por CPython que permite que, dentro de un mismo proceso, solo un hilo ejecute bytecode de Python a la vez.

Su función principal es proteger el estado interno del intérprete y simplificar la gestión de memoria, especialmente el recuento de referencias de los objetos. No elimina las condiciones de carrera de la aplicación: si varios hilos modifican una estructura compartida, todavía pueden ser necesarios locks u otros mecanismos de sincronización.

El principal efecto del GIL es que varios threads no suelen acelerar un cálculo intensivo escrito en Python puro. Aunque el equipo tenga varios núcleos, los hilos se alternan para ejecutar bytecode en lugar de hacerlo simultáneamente.

En mi sistema, esto afecta de forma distinta según la tarea:

- La ingesta WebSocket y las peticiones REST son I/O-bound, por lo que el GIL no constituye el principal cuello de botella. La mayor parte del tiempo se espera a la red.
- La escritura en SQLite también es principalmente I/O-bound, por lo que podría trasladarse a un thread.
- El cálculo de estrategias sobre grandes volúmenes de datos podría convertirse en CPU-bound. Si fuese demasiado costoso, lo trasladaría a procesos separados.

Las operaciones de pandas y NumPy pueden ejecutar parte de su trabajo en código nativo y, en determinadas operaciones, liberar el GIL. Sin embargo, no asumiría que cualquier cálculo con pandas escala automáticamente con threads: mediría primero mediante profiling.

6.1.3. Evitar que una función bloqueante congele el event loop

Una función bloqueante no debe ejecutarse directamente dentro del event loop si puede tardar un tiempo significativo. Mientras esa función está ejecutándose, ninguna otra corrutina del mismo loop puede avanzar.

Por ejemplo, actualmente el consumidor realiza una inserción síncrona:

```
inserted = insert_trade(
    conn,
    symbol=trade.symbol,
    trade_id=trade.trade_id,
    price=trade.price,
    qty=trade.qty,
    event_time_ms=trade.event_time_ms,
    is_buyer_maker=trade.is_buyer_maker,
)
```

La propia implementación reconoce que sqlite3 es síncrono y que, con un caudal elevado, sería necesario sacarlo del event loop o utilizar inserciones por lotes.

Una solución sería ejecutar la función bloqueante en un thread mediante asyncio.to_thread():

```
inserted = await asyncio.to_thread(
    insert_trade,
    conn,
    trade.symbol,
    trade.trade_id,
```

```
    trade.price,
    trade.qty,
    trade.event_time_ms,
    trade.is_buyer_maker,
)
```

De esta forma, la operación bloqueante se ejecuta en un hilo auxiliar y el event loop puede continuar atendiendo el WebSocket y otras corrutinas.

No obstante, con SQLite tendría que respetar las restricciones de uso de las conexiones entre threads. La opción más segura sería encapsular la creación de la conexión y la escritura dentro de la función ejecutada en el thread, o mantener un único thread escritor propietario de su conexión.

Otra mejora sería insertar los trades por lotes:

```
100 trades → una transacción y un commit
```

en lugar de:

```
100 trades → 100 transacciones y 100 commits
```

Si la función bloqueante fuese CPU-bound, asyncio.to_thread() evitaría congelar el loop, pero no proporcionaría un paralelismo significativo debido al GIL. En ese caso utilizaría un ProcessPoolExecutor.

Por tanto, la estrategia dependería del tipo de bloqueo:

```
Función bloqueante de I/O  → asyncio.to_thread()
Función intensiva de CPU   → ProcessPoolExecutor
Librería con API async     → await directamente
```

6.2. Avanzadas

- Diseña un pipeline concurrente para ingesta, almacenamiento y cálculo usando asyncio.Queue.
- Explica una situación donde await puede introducir un race condition y cómo evitarlo.
- Explica cómo medir y optimizar la latencia del event loop.

6.2.1. Pipeline concurrente con asyncio.Queue

Mi implementación actual ya utiliza un pipeline productor-consumidor:

```
WebSocket → producer → asyncio.Queue → consumer → SQLite
```

El productor recibe los mensajes de Binance, los transforma en objetos Trade y los introduce en una cola acotada. El consumidor extrae cada trade, valida su monotonicidad y lo persiste en SQLite.

Para separar también el cálculo de métricas y estrategias, ampliaría el diseño a tres etapas:

WebSocket

```
    ↓
```

queue_raw

```
    ↓
```

normalización y validación

```
    ↓
```

queue_storage

```
    ↓
```

persistencia por lotes

```
    ↓
```

queue_calculation

```
    ↓
```

cálculo de velas, señales y métricas

Una versión simplificada sería:

```
async def producer(raw_queue):
    async for message in websocket:
        trade = parse_trade(message)
        await raw_queue.put(trade)

async def validator(raw_queue, storage_queue):
    while True:
        trade = await raw_queue.get()

        monotonicity_validator.check(
            trade.symbol,
            trade.event_time_ms,
        )

        await storage_queue.put(trade)
        raw_queue.task_done()

async def storage_worker(storage_queue, calculation_queue):
    batch = []

    while True:
        trade = await storage_queue.get()
        batch.append(trade)
        storage_queue.task_done()
```

```
        if len(batch) >= 100:
            await asyncio.to_thread(insert_batch, batch)
            await calculation_queue.put(batch.copy())
            batch.clear()

async def calculation_worker(calculation_queue):
    while True:
        batch = await calculation_queue.get()

        await asyncio.to_thread(
            update_candles_and_metrics,
            batch,
        )

        calculation_queue.task_done()
```

Todas las colas serían acotadas para evitar un crecimiento ilimitado de memoria. Cada etapa tendría sus propias métricas de profundidad y lag, lo que permitiría localizar el cuello de botella.

Mantendría un único escritor de SQLite para evitar contención entre conexiones. Las inserciones se realizarían por lotes y fuera del event loop. Los cálculos intensivos podrían trasladarse a un ProcessPoolExecutor si el profiling mostrase que son CPU-bound.

Este diseño separa claramente:

```
Recepción           → I/O asíncrono
Validación          → trabajo ligero en el event loop
Persistencia        → I/O bloqueante en un thread
Cálculo intensivo   → procesos si es CPU-bound
```

6.2.2. Race condition introducida por un await

Aunque asyncio normalmente utiliza un solo hilo, puede haber condiciones de carrera. Cada await es un punto donde la corrutina cede el control y otra tarea puede modificar el estado compartido.

Por ejemplo:

```
if symbol not in positions:
    await load_position(symbol)
    positions[symbol] = Position(symbol)
```

Dos corrutinas podrían comprobar simultáneamente que symbol no existe. Ambas cederían el control durante await load_position(), cargarían el mismo estado y después escribirían sobre el diccionario. Si además aplicasen fills distintos, una actualización podría sobrescribir a la otra.

Otro ejemplo sería:

```
current = counters["inserted"]
await persist_trade(trade)
counters["inserted"] = current + 1
```

Entre la lectura y la escritura del contador, otra corrutina podría modificarlo. El resultado final perdería una actualización.

Una solución es proteger la sección crítica con asyncio.Lock:

```
lock = asyncio.Lock()

async with lock:
    current = counters["inserted"]
    await persist_trade(trade)
    counters["inserted"] = current + 1
```

No obstante, es preferible evitar mantener el lock durante una operación de red o disco, porque bloquearía a las demás tareas durante demasiado tiempo. Una opción mejor sería persistir primero y proteger únicamente la actualización compartida:

```
inserted = await persist_trade(trade)

async with lock:
    if inserted:
        counters["inserted"] += 1
```

En mi pipeline actual este riesgo se reduce porque existe un solo consumidor. Solo esa corrutina actualiza las estadísticas, valida la monotonicidad y escribe en SQLite. Por tanto, no hay varios consumidores modificando simultáneamente esos estados.

Otra forma de evitar carreras es aplicar el modelo de **propiedad exclusiva del estado**: cada estructura mutable pertenece a una única tarea y las demás se comunican con ella mediante colas. En muchos casos es más sencillo y seguro que compartir diccionarios protegidos por locks.

6.2.3. Medición de la latencia del event loop

La latencia del event loop es el retraso entre el instante en que una tarea debería ejecutarse y el instante en que realmente consigue hacerlo.

Puede medirse mediante una corrutina periódica:

```
import asyncio
import statistics

async def monitor_event_loop(samples, interval=0.1):
    loop = asyncio.get_running_loop()
    expected = loop.time() + interval

    while True:
        await asyncio.sleep(interval)

        actual = loop.time()
        lag_ms = max(0.0, actual - expected) * 1000
        samples.append(lag_ms)

        expected = actual + interval
```

Si el loop no está bloqueado, el retraso será pequeño. Si una función síncrona tarda 200 ms, el monitor también se ejecutará tarde y registrará un pico cercano a esa magnitud.

Calcularía:

```
p50 → retraso habitual
```

```
p95 → comportamiento en carga elevada
p99 → peores bloqueos del loop
máximo → pico extremo observado
```

El proyecto ya utiliza percentiles p50, p95 y p99 para medir la duración de funciones mediante el decorador measure_latency. El mismo enfoque puede aplicarse a las muestras de lag del event loop.

También activaría el modo debug de asyncio:

```
loop = asyncio.get_running_loop()
loop.set_debug(True)
loop.slow_callback_duration = 0.05
```

Esto permite detectar callbacks o tareas que bloquean el loop durante más de 50 ms.

Las métricas del event loop deberían relacionarse con:

- profundidad de las colas;
- throughput de entrada y salida;
- duración de las operaciones SQLite;
- tiempo de procesamiento por trade;
- uso de CPU;
- número de reconexiones.

Por ejemplo, si aumentan simultáneamente el lag del loop y la duración de las inserciones, el probable cuello de botella es la persistencia síncrona.

6.2.4. Optimización de la latencia

Aplicaría las optimizaciones en este orden:

1. **Eliminar operaciones bloqueantes del event loop.** La escritura con sqlite3 se trasladaría a asyncio.to_thread().
2. **Insertar por lotes.** Un solo commit para varios trades reduce el número de accesos al disco.
3. **Mantener pequeñas las secciones críticas.** No realizaría await dentro de un lock salvo que fuese imprescindible.
4. **Separar I/O y CPU.** La red permanecería en asyncio; los cálculos intensivos pasarían a procesos.
5. **Utilizar colas acotadas entre etapas.** Esto evita que una etapa lenta provoque un consumo ilimitado de memoria.
6. **Particionar por símbolo.** Los cálculos de distintos activos pueden ejecutarse de forma independiente.

7. **Medir antes de optimizar.** Utilizaría percentiles, profiling y métricas de cola para identificar la ruta crítica real.

En la implementación actual, la principal mejora sería sustituir el commit por trade por persistencia en lotes y sacar SQLite del event loop. El propio consumidor documenta que la llamada síncrona es aceptable para el caudal observado, pero no para miles de trades por segundo.

7. Uso avanzado de Python: decoradores, metaclases y dunder methods

7.1. Básicas

- ¿Qué responsabilidad cumple el decorador personalizado que exigía la práctica? Explica su efecto real.
- Explica qué ventaja aporta una metaclase en tu diseño concreto.
- Menciona dos dunder methods usados y explica cómo mejoran la semántica de tus objetos.

7.1.1. Responsabilidad del decorador personalizado

El decorador personalizado del proyecto es @measure_latency. Su responsabilidad es medir cuánto tarda en ejecutarse una función crítica sin modificar su comportamiento ni su interfaz pública.

El decorador registra el tiempo con time.perf_counter() y guarda las mediciones en una ventana acotada de las últimas 10.000 llamadas. Posteriormente, latency_report() calcula el número de observaciones y los percentiles p50, p95 y p99.

Está diseñado para funcionar tanto con funciones síncronas como asíncronas. En el caso asíncrono, el wrapper también debe ser async y ejecutar:

```
return await func(*args, **kwargs)
```

De lo contrario, solo mediría el tiempo que se tarda en crear la corrutina, no su ejecución real.

También utiliza:

```
@functools.wraps(func)
```

para conservar el nombre, la documentación y la firma aparente de la función original. Esto es importante para la introspección, el debugging y frameworks como FastAPI.

El registro se realiza dentro de un bloque finally, por lo que también se mide la duración de las llamadas que terminan con una excepción.

El decorador tiene un efecto real en el sistema, ya que se aplica al cálculo de señales de las tres estrategias y al cálculo del retorno del portafolio:

```
@measure_latency
def generate_signals(...):
    ...
@measure_latency
def portfolio_returns(...):
    ...
```

Las mediciones acumuladas se exponen mediante el endpoint /metrics de FastAPI, por lo que el decorador no es únicamente un ejemplo aislado, sino que forma parte de la observabilidad del backend.

7.1.2. Ventaja de la metaclase en el diseño

La metaclase StrategyMeta cumple dos responsabilidades concretas:

1. Validar el contrato de las estrategias.
2. Registrar automáticamente cada estrategia válida.

Cuando se define una subclase de Strategy, la metaclase comprueba que incluya:

un atributo de clase name de tipo str

un método generate_signals() implementado

Si falta alguno de estos elementos, se lanza un TypeError durante la definición de la clase, es decir, al importar el módulo. Esto permite detectar una estrategia mal implementada antes de arrancar el sistema o intentar instanciarla.

La segunda responsabilidad es introducir automáticamente la clase en:

STRATEGY_REGISTRY

con la estructura:

```
nombre de estrategia → clase
```

Por ejemplo:

```
"sma_cross" → SMACrossStrategy
"momentum"  → MomentumStrategy
"engulfing" → EngulfingStrategy
```

La ventaja concreta es que el backend no necesita importar ni conocer cada estrategia individualmente. Consulta el registro y crea la estrategia seleccionada:

```
strategy = STRATEGY_REGISTRY[strategy_name]()
```

Añadir una nueva estrategia consiste principalmente en definir una nueva subclase válida. No es necesario modificar manualmente una lista central ni cambiar los endpoints de la API. Esto reduce el acoplamiento y aplica el principio abierto/cerrado: el sistema está abierto a nuevas estrategias, pero no necesita modificar su lógica principal para incorporarlas.

7.1.3. Dunder methods utilizados

Uno de los dunder methods utilizados es Trade.___eq___(). Define que dos trades son iguales cuando comparten su identidad de mercado:

(symbol, trade_id)

No se comparan todos sus atributos. Por ejemplo, el mismo trade recibido dos veces puede tener distinto ingestion_time, pero sigue representando la misma operación del exchange.

```
def __eq__(self, other):
    return (self.symbol, self.trade_id) == (
        other.symbol,
        other.trade_id,
```

```
    )
```

Esto mejora la semántica porque la igualdad representa identidad y no una coincidencia accidental de todos los datos.

El segundo es Trade.___hash___(), que utiliza exactamente los mismos campos que ___eq___():

```
def __hash__(self):
    return hash((self.symbol, self.trade_id))
```

Así se cumple el contrato de Python:

si a == b, entonces hash(a) == hash(b)

Esto permite utilizar objetos Trade correctamente en conjuntos y diccionarios. Si el mismo trade se recibe dos veces, un set conserva una sola entrada. La clase es además inmutable mediante frozen=True, evitando que cambien los campos usados en el hash después de introducir el objeto en un conjunto.

Otro dunder relevante es Position.___add___(), que permite expresar la actualización de una posición de forma natural:

```
new_position = position + fill
```

El método no realiza una suma numérica convencional. Aplica la semántica financiera del fill: aumenta, reduce, cierra o invierte la posición y devuelve un nuevo objeto Position.

Por ejemplo:

```
position = Position("BTCUSDT", qty=10, avg_price=100)
fill = Fill("BTCUSDT", qty=-5, price=120)

new_position = position + fill
```

El resultado es una posición larga de 5 unidades, con precio medio 100 y un PnL realizado de 100.

Estos métodos mejoran la semántica de los objetos porque permiten que las operaciones propias de Python representen directamente conceptos del dominio:

```
trade_a == trade_b → misma identidad de mercado
hash(trade)        → deduplicación en sets y diccionarios
position + fill    → transición al siguiente estado de la posición
```

7.2. Avanzadas

- Diseña un decorador que mida latencia de funciones críticas sin alterar su firma.
- Explica cómo una metaclase puede validar automáticamente la implementación de un puerto hexagonal.
- Implementa un ejemplo conceptual donde ___enter___ y ___exit___ garantizan consistencia transaccional sin usar SQL.

7.2.1. Decorador para medir latencia sin alterar la firma

El proyecto implementa el decorador @measure_latency, que mide la duración de funciones críticas tanto síncronas como asíncronas. Para no alterar la identidad de la función original utiliza functools.wraps, que conserva atributos como ___name___, ___doc___ y la firma utilizada por herramientas de introspección.

```
import functools
import inspect
import time
from collections import defaultdict, deque
from collections.abc import Callable

_LATENCIES = defaultdict(lambda: deque(maxlen=10_000))

def measure_latency(func: Callable) -> Callable:
    name = func.__qualname__

    if inspect.iscoroutinefunction(func):

        @functools.wraps(func)
        async def async_wrapper(*args, **kwargs):
            start = time.perf_counter()
            try:
                return await func(*args, **kwargs)
            finally:
                elapsed_ms = (time.perf_counter() - start) * 1000
                _LATENCIES[name].append(elapsed_ms)

        return async_wrapper

    @functools.wraps(func)
    def sync_wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            elapsed_ms = (time.perf_counter() - start) * 1000
            _LATENCIES[name].append(elapsed_ms)

    return sync_wrapper
```

La distinción entre funciones síncronas y asíncronas es necesaria. Si una función async se envolviese con un wrapper normal, solo se mediría el tiempo empleado en crear la corrutina, no el tiempo de su ejecución. Por eso el wrapper asíncrono debe hacer await func(...).

El uso de finally garantiza que también se registre la latencia de las llamadas que terminan con una excepción. La colección utiliza una deque acotada para evitar que el proceso acumule mediciones indefinidamente.

7.2.2. Validación de un puerto hexagonal mediante una metaclase

En una arquitectura hexagonal, un **puerto** define el contrato que deben cumplir los adaptadores externos. Por ejemplo, un puerto de exchange podría exigir un nombre de adaptador y un método asíncrono para descargar velas.

Una metaclase puede comprobar ese contrato cuando se define la clase, antes de que el sistema

intente utilizarla:

```
import inspect

class ExchangeAdapterMeta(type):
    def __new__(mcs, name, bases, namespace):
        cls = super().__new__(mcs, name, bases, namespace)

        # La clase base representa el puerto y no se valida.
        if not bases:
            return cls

        if not isinstance(namespace.get("name"), str):
            raise TypeError(
                f"{name} debe declarar un atributo 'name' de tipo str"
            )

        method = namespace.get("fetch_klines")

        if method is None or not inspect.iscoroutinefunction(method):
            raise TypeError(
                f"{name} debe implementar async fetch_klines()"
            )

        return cls

class ExchangePort(metaclass=ExchangeAdapterMeta):
    name = ""

    async def fetch_klines(
        self,
        symbol: str,
        interval: str,
        start_ms: int,
    ) -> list:
        raise NotImplementedError
```

Un adaptador válido sería:

```
class BinanceAdapter(ExchangePort):
    name = "binance"

    async def fetch_klines(
        self,
        symbol: str,
        interval: str,
        start_ms: int,
    ) -> list:
        return []
```

Mientras que esta clase sería rechazada al importarse:

```
class BrokenAdapter(ExchangePort):
    name = "broken"
    # Falta fetch_klines()
```

La ventaja es el comportamiento *fail-fast*: un adaptador mal construido no llega a ejecutarse ni a registrarse.

Esta es la misma idea aplicada en mi proyecto mediante StrategyMeta. La metaclase comprueba que cada estrategia declare un atributo name y un método generate_signals(), y después la registra automáticamente en STRATEGY_REGISTRY.

Una limitación es que una metaclase valida principalmente la estructura del contrato: puede comprobar que el método existe y que es invocable, pero no garantiza por sí sola que su comportamiento sea correcto. Para eso siguen siendo necesarios tests de contrato e integración.

7.2.3. Consistencia transaccional con ___enter___ y ___exit___ sin SQL

Es posible implementar una transacción sobre archivos utilizando un context manager. La idea es trabajar sobre una copia temporal y sustituir el archivo original únicamente si todas las operaciones terminan correctamente.

```
import json
import os
import tempfile
from copy import deepcopy
from pathlib import Path

class FileTransaction:
    def __init__(self, path: Path) -> None:
        self.path = path
        self.original_data = {}
        self.working_data = {}
        self.temp_path: Path | None = None

    def __enter__(self) -> dict:
        # Cargar el último estado confirmado.
        if self.path.exists():
            with self.path.open("r", encoding="utf-8") as file:
                self.original_data = json.load(file)
        else:
            self.original_data = {}

        # Todas las modificaciones se realizan sobre una copia.
        self.working_data = deepcopy(self.original_data)
        return self.working_data

    def __exit__(self, exc_type, exc_value, traceback) -> bool:
        # Si ocurrió una excepción, se descarta el estado provisional.
        if exc_type is not None:
            self.working_data = deepcopy(self.original_data)
            return False  # La excepción se propaga.

        self.path.parent.mkdir(parents=True, exist_ok=True)

        fd, temp_name = tempfile.mkstemp(
            dir=self.path.parent,
            prefix=f"{self.path.name}.",
            suffix=".tmp",
        )
        self.temp_path = Path(temp_name)

        try:
```

```
            with os.fdopen(fd, "w", encoding="utf-8") as file:
                json.dump(self.working_data, file)
                file.flush()
                os.fsync(file.fileno())

            # Sustitución atómica: se ve el archivo antiguo o el nuevo,
            # pero no un archivo parcialmente escrito.
            os.replace(self.temp_path, self.path)

        finally:
            if self.temp_path.exists():
                self.temp_path.unlink()

        return False
```

Su uso sería:

```
portfolio_file = Path("data/positions.json")

with FileTransaction(portfolio_file) as positions:
    positions["BTCUSDT"] = {
        "qty": 10,
        "avg_price": 60_000,
    }

    positions["ETHUSDT"] = {
        "qty": 5,
        "avg_price": 3_000,
    }
```

Si todas las instrucciones del bloque terminan correctamente, ___exit___ escribe el nuevo estado en un archivo temporal y ejecuta os.replace(). Esta operación reemplaza atómicamente el archivo anterior.

Si ocurre una excepción:

```
with FileTransaction(portfolio_file) as positions:
    positions["BTCUSDT"]["qty"] = 20
    raise RuntimeError("Fallo durante la actualización")
```

___exit___ detecta el error y no reemplaza el archivo original. Por tanto, el estado queda completamente actualizado o completamente sin modificar:

```
éxito     → commit del estado completo
excepción → rollback del estado provisional
```

Este ejemplo proporciona atomicidad frente a errores durante el bloque y frente a escrituras parciales. Para soportar varios procesos concurrentes sería necesario añadir además un bloqueo de archivo, porque el atomic rename evita archivos corruptos, pero por sí solo no impide que dos escritores partan del mismo estado y sobrescriban mutuamente sus cambios.

8. Escalado horizontal y multiactivo

8.1. Básicas

- Explica qué rompe cuando pasas de 10 a 300 activos en tu sistema actual.
- Indica los límites de CPU, IO y memoria relevantes.

8.1.1. ¿Qué se rompe al pasar de 10 a 300 activos?

El sistema actual está diseñado para un universo pequeño de diez activos. Al pasar a 300, varias decisiones que ahora son razonables dejarían de escalar correctamente.

En primer lugar, todos los streams se concentran en una única conexión WebSocket combinada:

```
300 streams → una sola conexión → una sola cola → un solo consumidor
```

Con diez activos, esta solución es sencilla y facilita la reconexión. Con 300, aumenta mucho el caudal total y una caída del socket afecta simultáneamente a todo el universo. También crece el tiempo necesario para recuperar los datos perdidos después de una desconexión. La propia implementación indica que, con cientos de símbolos, sería necesario repartirlos entre varias conexiones.

El siguiente límite es la cola:

```
asyncio.Queue(maxsize=1000)
```

Su capacidad no depende del número de activos. Si el volumen agregado aumenta treinta veces, esos 1.000 elementos representan un margen temporal mucho menor. La cola se llenaría con más frecuencia, bloquearía al productor en await queue.put() y aumentaría el riesgo de dejar de leer el socket a tiempo.

También existe un único consumidor que valida y persiste todos los trades. Además, insert_trade() utiliza SQLite de forma síncrona y realiza un commit por evento. Con 300 activos, este escritor único probablemente se convertiría en el principal cuello de botella.

El cálculo del PnL también escala aproximadamente de forma lineal con el número de símbolos. portfolio_returns() recorre todos los activos, carga sus velas desde SQLite, genera señales y construye un DataFrame conjunto. Con 300 activos, cada petición al endpoint /pnl implicaría muchas más consultas, cálculos y memoria temporal.

Además, el listado de activos está definido estáticamente en varias partes del código mediante una lista SYMBOLS. Para integrar cientos o miles de símbolos convendría obtener el universo desde configuración o almacenamiento, evitando mantener listas duplicadas en los módulos de ingesta y PnL. Por último, el cliente REST limita la concurrencia con un semáforo de tres descargas simultáneas. Esta protección evita superar los límites de Binance, pero descargar el histórico de 300 activos tardaría mucho más y requeriría controlar cuidadosamente la paginación, los pesos del rate limit y la reanudación de descargas incompletas.

8.1.2. Límites de CPU

Las principales cargas de CPU serían:

- cálculo de medias móviles, momentum y patrones de velas;

- generación de señales para todos los activos;
- alineación de cientos de series temporales con pandas;
- cálculo del portafolio y composición del PnL;
- serialización de grandes respuestas de la API;
- renderizado de gráficos y tablas en el dashboard.

Las estrategias actuales realizan operaciones vectorizadas con pandas, pero el coste sigue creciendo con el número de activos y de velas. El endpoint de PnL vuelve a calcular las señales sobre todo el histórico disponible cada vez que se solicita. Con 300 activos, sería preferible calcular de forma incremental y almacenar resultados intermedios.

El GIL no es el principal problema en la recepción del WebSocket, que es I/O-bound, pero podría limitar el paralelismo de cualquier transformación intensiva escrita en Python puro. Los cálculos CPU-bound deberían particionarse por símbolo y ejecutarse en procesos separados cuando el profiling lo justificase.

8.1.3. Límites de I/O

Los límites de entrada y salida más relevantes son:

1. **Red del WebSocket:** aumenta el número total de mensajes recibidos por segundo.
2. **Rate limits de REST:** descargar o recuperar el histórico de 300 activos requiere muchas más peticiones.
3. **Escritura en SQLite:** existe un único escritor efectivo y actualmente se realiza un commit por trade.
4. **Lecturas del backend:** el cálculo del portafolio ejecuta consultas independientes por símbolo.
5. **Disco:** el tamaño de las tablas y del archivo WAL aumenta de forma continua.

SQLite en modo WAL permite lecturas mientras se escribe, pero no convierte la base de datos en un sistema con múltiples escritores paralelos. Bajo carga elevada aparecerían más esperas, mayor tamaño del WAL y posibles errores de bloqueo.

Antes de migrar de tecnología, optimizaría la implementación mediante:

inserciones por lotes

una transacción por lote

índices adecuados

cálculo incremental

caché de señales y PnL

Si aun así el caudal superase la capacidad de SQLite, migraría a PostgreSQL o a un sistema de almacenamiento particionado.

8.1.4. Límites de memoria

El uso de memoria aumentaría principalmente por:

- eventos pendientes en las colas;
- buffers de reordenación por símbolo;
- DataFrames históricos cargados para cada activo;
- unión de cientos de series en el cálculo del portafolio;
- caché del dashboard;
- resultados temporales de estrategias;
- métricas e instrumentación.

La cola está acotada, por lo que no puede crecer indefinidamente. Sin embargo, con 300 activos podría permanecer cerca de su capacidad máxima y aumentar mucho el lag.

El cálculo:

```
pd.DataFrame(per_asset)
```

mantiene simultáneamente en memoria las series de retornos de todos los activos. Con más símbolos y más histórico, esta matriz puede crecer considerablemente.

La instrumentación de latencia utiliza una deque(maxlen=10_000) por función, por lo que sus muestras sí están acotadas y no generan una fuga de memoria ilimitada.

Para escalar reduciría la memoria mediante:

- procesamiento por ventanas temporales;
- cálculo incremental;
- liberación de DataFrames intermedios;
- partición de activos entre workers;
- buffers acotados por símbolo;
- almacenamiento de métricas agregadas en lugar de recalcular todo el histórico.

En resumen, al pasar de 10 a 300 activos no se rompe la lógica del dominio, pero sí las decisiones de infraestructura: una sola conexión, una única cola, un consumidor, commits individuales, consultas secuenciales y cálculos completos por petición. La solución sería particionar el trabajo por símbolo, realizar persistencia por lotes y desacoplar ingesta, almacenamiento y cálculo.

8.2. Avanzadas

- Explica cómo particionar el procesamiento por símbolo para evitar contención.
- Diseña una arquitectura con varias réplicas del backend sin duplicar trabajo.
- Explica cómo preparar el sistema para miles de activos sin reescribir el dominio.

8.2.1. Particionado del procesamiento por símbolo

Para evitar contención, dividiría el universo de activos en particiones y asignaría cada símbolo siempre al mismo worker. La regla de reparto podría ser:

```
partición = hash(symbol) % número_de_workers
```

Por ejemplo:

```
Worker 0 → BTCUSDT, ADAUSDT, LINKUSDT
Worker 1 → ETHUSDT, BNBUSDT, AVAXUSDT
Worker 2 → XRPUSDT, SOLUSDT, DOGEUSDT
```

Cada partición tendría su propia conexión WebSocket o subconjunto de streams, su asyncio.Queue acotada y su estado de monotonicidad. De esta forma, todos los eventos de un símbolo se procesan secuencialmente en el mismo worker y no es necesario proteger continuamente su estado con locks.

```
Streams partición 0 → queue_0 → worker_0
Streams partición 1 → queue_1 → worker_1
Streams partición 2 → queue_2 → worker_2
```

La afinidad por símbolo es importante porque operaciones como la validación temporal, la construcción de velas y el cálculo de indicadores dependen del orden de los eventos anteriores del mismo activo. En cambio, BTC y ETH pueden procesarse en paralelo porque sus estados son independientes.

El sistema actual ya mantiene la monotonicidad por símbolo mediante un diccionario separado para cada activo. El particionado extendería esa misma idea a nivel de workers.

Para evitar desequilibrios, no repartiría simplemente el mismo número de símbolos a cada worker. BTC puede generar muchos más trades que un activo menos líquido. Mediría el throughput por símbolo y redistribuiría las particiones cuando un worker concentrase una carga excesiva.

La persistencia podría seguir dos modelos:

- Un único escritor por lotes que reciba eventos de todos los workers.
- Una partición de almacenamiento por símbolo o grupo de símbolos.

Con SQLite mantendría un único escritor para evitar contención. Con PostgreSQL podría admitir varios escritores, manteniendo la restricción única sobre (symbol, trade_id).

8.2.2. Varias réplicas del backend sin duplicar trabajo

El backend actual no mantiene estado propio. Cada petición abre una conexión, lee los datos persistidos, calcula la respuesta y cierra la conexión. Esto facilita crear varias réplicas porque ninguna depende de la memoria de otra.

La arquitectura sería:

```
                  +→ Backend 1 -+
Cliente → balanceador → Backend 2 -+→ almacenamiento compartido
                  +→ Backend 3 -+
```

Las réplicas serían principalmente de lectura. Un balanceador distribuiría las peticiones entre ellas y todas consultarían la misma fuente de verdad.

Con el archivo SQLite local actual no sería recomendable ejecutar las réplicas en hosts distintos, porque cada máquina tendría su propia copia del archivo o necesitaría compartirlo mediante un sistema de archivos de red. Para escalar horizontalmente migraría la persistencia a una base de datos compartida, como PostgreSQL.

Para evitar duplicar cálculos costosos, separaría el backend HTTP de los workers de cálculo:

```
Ingestores → almacenamiento
                  ↓
          workers de cálculo
                  ↓
          métricas persistidas o caché
                  ↓
        réplicas de FastAPI
```

Las réplicas de FastAPI no recalcularían continuamente las mismas estrategias. Leerían señales y PnL previamente calculados o almacenados en una caché compartida.

Si los cálculos se ejecutasen mediante una cola de trabajos, cada tarea tendría una identidad estable:

(strategy, symbol, interval, candle_close_time)

Los workers consumirían las tareas mediante un mecanismo de reparto exclusivo, como un grupo de consumidores. Solo un worker recibiría cada tarea. Además, la clave de la tarea se almacenaría con una restricción única, de modo que un reintento no generase dos resultados.

Esta arquitectura combina:

- reparto de trabajo mediante la cola;
- idempotencia mediante claves únicas;
- resultados compartidos mediante base de datos o caché;
- réplicas HTTP sin estado.

La ingesta WebSocket tampoco se ejecutaría dentro de cada réplica del backend. Si tres réplicas abriesen los mismos streams, recibirían y procesarían tres veces los mismos eventos. Mantendría la ingesta como un servicio independiente y particionado, mientras que FastAPI actuaría únicamente como capa de consulta.

8.2.3. Preparación para miles de activos sin reescribir el dominio

El dominio debe permanecer independiente del número de activos y de la infraestructura utilizada. Las entidades Trade, Fill y Position operan sobre un símbolo, pero no conocen Binance, SQLite, WebSockets ni el número total de activos. Por ello, el escalado debería resolverse en los adaptadores y en la orquestación:

Dominio:

Trade, Position, Fill, estrategias

Infraestructura:

WebSockets, colas, particiones, base de datos, caché, workers

Para preparar el sistema para miles de activos realizaría los siguientes cambios:

1. **Universo dinámico de símbolos**

Eliminaría las listas SYMBOLS duplicadas en el código. Los activos se obtendrían desde configuración o desde una tabla de instrumentos.

2. **Interfaces para los adaptadores**

Definiría puertos para exchange, almacenamiento y publicación de eventos. El dominio recibiría objetos normalizados y no conocería si proceden de Binance, otro exchange o un fichero de pruebas.

3. **Particionado estable**

Asignaría cada símbolo a una partición mediante hashing consistente. Así podría añadir o retirar workers moviendo solo una parte de los símbolos.

4. **Procesamiento incremental**

No recalcularía todo el histórico cada vez. Cada vela nueva actualizaría únicamente los indicadores, señales y métricas afectados.

5. **Almacenamiento particionado**

Los datos podrían particionarse por fecha y símbolo. Esto reduciría el volumen leído por consulta y permitiría repartir la carga entre nodos.

6. **Configuración externa**

Ventanas, estrategias, activos y número de particiones dejarían de estar codificados directamente en los módulos.

7. **Mensajería desacoplada**

Entre ingesta y cálculo utilizaría un broker o log distribuido. Los consumidores podrían escalar independientemente y reconstruir su estado reproduciendo eventos.

8. **Observabilidad por partición**

Mediría throughput, lag, errores y uso de recursos por worker y por símbolo para detectar activos especialmente costosos.

La metaclase y el registro de estrategias ya ayudan a mantener extensible la lógica cuantitativa: una nueva estrategia válida se registra sin modificar el backend ni una lista central de clases.

Por tanto, pasar de diez a miles de activos no debería exigir cambiar las reglas de Position, Trade o las estrategias. El cambio principal estaría en cómo se distribuyen, almacenan y procesan los eventos. El objetivo es escalar la infraestructura sin alterar la semántica del dominio.

9. Tolerancia a fallos y recuperación determinista

9.1. Básicas

- ¿Tu sistema garantiza at least once, at most once o exactly once? Justifícalo.
- Define qué ocurre si la persistencia falla temporalmente.

9.1.1. Garantía de entrega del sistema

El sistema actual no ofrece una garantía completa de exactly once de extremo a extremo. Conviene separar la **entrega del evento** de su **efecto en el almacenamiento**.

En el WebSocket, un trade puede volver a recibirse tras una reconexión. Sin embargo, SQLite utiliza como clave primaria:

(symbol, trade_id)

y la inserción se realiza mediante:

```
INSERT OR IGNORE
```

Por tanto, aunque un evento se procese varias veces, solo puede existir una fila para esa identidad. El efecto persistido es como máximo una inserción por trade.

No obstante, la implementación no mantiene un offset confirmado, no solicita a Binance la repetición de los mensajes pendientes y no ejecuta automáticamente un backfill REST después de una desconexión. Si un trade se pierde durante la caída de la conexión o antes de persistirse, el sistema actual no garantiza recuperarlo.

Por ello, la clasificación más precisa es:

Entrega del WebSocket:

best effort; no garantiza at least once.

Efecto en SQLite:

at most once por (symbol, trade_id), gracias a la clave única.

Sistema completo:

no garantiza exactly once.

La semántica deseable sería **at least once con procesamiento idempotente**: permitir que los eventos se repitan durante reconexiones o reintentos, pero hacer que su efecto se aplique una sola vez. Para conseguirlo completamente añadiría un backfill REST desde el último evento confirmado después de cada reconexión. La restricción única de la base de datos descartaría los eventos ya almacenados.

Esto suele denominarse effectively once: la entrega puede repetirse, pero el estado final es equivalente a haber procesado cada evento una sola vez.

9.1.2. Fallo temporal de la persistencia

En la implementación actual, insert_trade() ejecuta directamente la inserción y el commit. No existe una política de reintentos específica alrededor de la escritura en SQLite.

Si la persistencia falla, por ejemplo por un error de disco, un bloqueo prolongado de SQLite o falta de espacio, la excepción sale del consumidor. Como productor y consumidor se ejecutan mediante asyncio.gather(), el proceso de ingesta termina con error en lugar de continuar silenciosamente.

Los trades que todavía permaneciesen en asyncio.Queue solo existen en memoria. Si el proceso termina, se pierden. Tampoco existe actualmente un log persistente intermedio que permita reproducirlos después del reinicio.

El comportamiento actual es, por tanto:

Fallo de SQLite

```
    ↓
```

el consumidor lanza una excepción

```
    ↓
```

se detiene la ejecución del ingestor

```
    ↓
```

los eventos no persistidos de la cola pueden perderse

La ventaja es que el fallo es visible y no se continúa fingiendo que los datos han sido guardados. La desventaja es que la recuperación no es automática.

Para tolerar un fallo temporal añadiría:

1. Reintentos con backoff para errores transitorios de persistencia.
2. Una cola acotada para absorber interrupciones breves.
3. Persistencia por lotes para reducir el número de commits.
4. Un log append-only o broker duradero antes de escribir en la base de datos.
5. Backfill REST desde el último timestamp o identificador confirmado.
6. Una alerta si la cola se aproxima a su capacidad máxima.

El evento solo se marcaría como procesado después de confirmar la escritura. Si el proceso fallase antes de esa confirmación, el evento se reproduciría al reiniciar. La clave (symbol, trade_id) permitiría repetir la operación sin generar duplicados.

9.2. Avanzadas

- Diseña un mecanismo de recuperación determinista tras un fallo entre ingesta y persistencia.
- Explica cómo evitar corrupción de datos durante una caída abrupta.
- ¿Cómo buffers datos sin reventar memoria si el almacenamiento cae durante minutos?

9.2.1. Recuperación tras un fallo entre ingesta y persistencia

En la implementación actual, los trades recibidos se almacenan temporalmente en una asyncio.Queue y después el consumidor los inserta en SQLite. Si el proceso cae antes de persistir un evento, los elementos que continúan en la cola se pierden, porque la cola solo existe en memoria.

Para conseguir una recuperación determinista añadiría un **log duradero append-only** entre la ingesta y SQLite:

WebSocket

```
    ↓
```

log duradero local

```
    ↓
```

asyncio.Queue

```
    ↓
```

SQLite

El procedimiento sería:

1. Recibir y normalizar el trade.
2. Asignarle una secuencia local creciente.
3. Escribirlo en un log append-only y confirmar su escritura en disco.
4. Introducir en la cola una referencia al registro.
5. Persistir los trades por lotes en SQLite.
6. Confirmar la transacción de la base de datos.
7. Actualizar un offset persistido con la última secuencia confirmada.

El checkpoint podría contener:

```
last_committed_sequence = 150000
```

Si el proceso se detuviese entre los pasos 3 y 6, el trade permanecería en el log, aunque todavía no estuviese en SQLite. Al reiniciar, el sistema leería el checkpoint y reproduciría todos los registros posteriores:

```
reproducir secuencias > last_committed_sequence
```

La repetición es segura porque la tabla utiliza como clave primaria:

(symbol, trade_id)

y las inserciones se realizan con INSERT OR IGNORE. Si un trade ya había sido confirmado en SQLite, pero el proceso cayó antes de actualizar el checkpoint, se reproduciría, pero la base de datos lo ignoraría como duplicado.

Así, el estado final depende únicamente del contenido del log, del checkpoint y de la clave de identidad, no del punto exacto en el que se produjo el fallo.

El algoritmo de recuperación sería:

```
def recover(journal, checkpoint, connection):
    pending = journal.read_from(checkpoint + 1)

    for batch in make_batches(pending, size=500):
        insert_trades_idempotently(connection, batch)
        connection.commit()

        checkpoint = batch[-1].sequence
        save_checkpoint_atomically(checkpoint)
```

El checkpoint solo avanza después de confirmar la transacción. Nunca se marca un evento como procesado antes de que su escritura haya terminado correctamente.

Para recuperar huecos producidos mientras el WebSocket estuvo desconectado sería necesario consultar una fuente histórica del exchange desde el último evento confirmado. El cliente REST actual descarga velas, no el histórico exacto de trades, por lo que una recuperación exacta del flujo de ticks requeriría otro endpoint de Binance o un broker duradero.

9.2.2. Evitar corrupción durante una caída abrupta

SQLite se configura en modo WAL, donde las escrituras se realizan primero en un *write-ahead log*. Esto permite conservar la consistencia de la base de datos y mantener lecturas concurrentes mientras el ingestor escribe.

Además, cada lote debería escribirse dentro de una única transacción:

```
BEGIN
    insertar lote
COMMIT
```

Si el proceso cae antes del COMMIT, el lote no debe considerarse confirmado. Si cae después, puede reproducirse de forma segura gracias a la idempotencia.

Para proteger el log y el checkpoint aplicaría estas medidas:

- Escritura append-only: nunca modificar registros anteriores.
- Registros con tamaño y checksum para detectar una escritura incompleta.
- flush() y fsync() antes de considerar duradero un lote.
- Checkpoint escrito primero en un archivo temporal.
- Sustitución mediante os.replace(), que realiza un cambio atómico.
- Segmentación del log en varios archivos para no reescribir un fichero enorme.

Si una caída deja incompleto el último registro, al reiniciar se validan los checksums y se descarta únicamente la cola corrupta del último segmento. Los registros completos anteriores siguen siendo válidos.

El checkpoint también debe incluir su propia versión o checksum. De esta forma, durante la recuperación se utiliza el último checkpoint completamente escrito, nunca un valor parcial.

Para una exigencia de durabilidad superior cambiaría además la configuración de SQLite de synchronous=NORMAL a synchronous=FULL. La configuración actual favorece el equilibrio entre rendimiento y durabilidad; FULL reduciría el riesgo de perder el último commit ante un corte brusco de alimentación, a cambio de más latencia de escritura.

9.2.3. Buffering cuando el almacenamiento falla durante minutos

La cola actual está acotada a 1.000 elementos:

```
asyncio.Queue(maxsize=1000)
```

Esto evita consumir memoria sin límite, pero no es suficiente para mantener varios minutos de tráfico si SQLite deja de estar disponible. Cuando se llena, el productor queda bloqueado y deja temporalmente de leer del WebSocket.

Para soportar una caída prolongada utilizaría dos niveles:

Buffer rápido en memoria:

asyncio.Queue acotada

Buffer duradero:

log segmentado en disco

Los eventos se escribirían rápidamente en el log local y la cola contendría únicamente referencias o lotes pendientes de inserción. De esta forma, el volumen acumulado durante varios minutos consume disco, no memoria RAM.

El spool de disco también tendría límites explícitos:

máximo número de segmentos

máximo tamaño total en GB

máxima antigüedad del evento pendiente

Mediría al menos:

eventos pendientes

bytes pendientes en disco

edad del evento más antiguo

throughput de entrada

throughput de persistencia

número de reintentos

Mientras el almacenamiento estuviese caído, el escritor reintentaría con backoff. Cuando se recuperase, vaciaría el log por lotes hasta que el throughput de salida superase al de entrada.

Si el buffer de disco alcanzase su límite, no descartaría trades silenciosamente. Aplicaría backpressure, generaría una alerta crítica y cerraría ordenadamente la conexión. Después sería necesario reconstruir el hueco desde una fuente histórica.

La política sería:

Caída breve:

absorber con la cola en memoria.

Caída de varios minutos:

persistir temporalmente en el log de disco.

Disco próximo al límite:

alertar, detener la ingesta de forma controlada y planificar backfill.

Esta estrategia mantiene la memoria acotada, conserva los eventos ya recibidos y permite reproducir siempre el mismo log hasta obtener el mismo estado final.

10. Pruebas avanzadas y validación del sistema

10.1. Básicas

- Explica cómo escribiste tests unitarios para tus estrategias.
- ¿Qué es un fake exchange y para qué sirve en testing?

10.1.1. Tests unitarios de las estrategias

Los tests de las estrategias utilizan datos sintéticos y deterministas. En lugar de descargar precios reales, construyo pequeños DataFrame de velas donde puedo calcular manualmente cuál debería ser la señal correcta antes de ejecutar la estrategia.

La función auxiliar make_candles() crea velas con las columnas exigidas por el contrato:

open, high, low, close, volume

y un índice temporal ordenado. A partir de esos datos se ejecuta generate_signals() y se compara el resultado con la señal esperada.

Para el cruce de medias utilizo ventanas pequeñas, fast=2 y slow=3, en lugar de las ventanas de producción 20/50. La lógica que se prueba es la misma, pero los cálculos pueden comprobarse fácilmente a mano. Por ejemplo:

```
cierres = [10, 10, 10, 20, 30, 40]

SMA rápida = (30 + 40) / 2 = 35
SMA lenta  = (20 + 30 + 40) / 3 = 30
```

Como la media rápida está por encima de la lenta, la señal esperada es +1. También se prueban la señal corta, el periodo inicial sin datos suficientes y el rechazo de parámetros incorrectos.

Para momentum se construyen series donde el retorno respecto al lookback supera el umbral positivo, cae por debajo del negativo o permanece dentro de la zona muerta. Por ejemplo:

```
120 / 100 − 1 = 20 %
```

Con un umbral del 5 %, la señal esperada es +1.

Para el patrón envolvente se crean dos velas con aperturas y cierres conocidos. Se comprueba un patrón alcista, uno bajista y un caso donde la segunda vela sube, pero su cuerpo no envuelve completamente al anterior.

Los tests siguen la misma idea:

datos controlados

→ resultado calculado manualmente

→ ejecución de la estrategia

→ assert contra el resultado esperado

Esto evita depender de que un resultado de backtest parezca razonable. Si la señal obtenida no coincide con la calculada previamente, el test falla.

10.1.2. Fake exchange

Un **fake exchange** es una implementación de pruebas que mantiene la misma interfaz que el cliente real del exchange, pero devuelve datos fabricados y controlados sin realizar peticiones de red.

En el test del pipeline, FakeExchange implementa el método asíncrono:

```
async def fetch_klines(
    self,
    symbol: str,
    interval: str,
    start_ms: int,
```

) ->list[tuple]:

```
    ...
```

De esta forma, el código que consume el cliente no necesita saber si está trabajando con Binance o con el fake. Ambos respetan el mismo contrato.

El fake permite que los tests sean:

- **Deterministas:** siempre devuelve los mismos datos.
- **Rápidos:** no existe latencia de red.
- **Robustos:** funcionan sin conexión y aunque Binance esté caído.
- **Independientes de rate limits:** no consumen cuota de la API.

En el test del pipeline, el fake genera una serie alcista conocida. Después se utiliza el almacenamiento SQLite real, la consulta real con pandas y la estrategia real de momentum:

FakeExchange

→ filas sintéticas

→ SQLite real

→ DataFrame real

→ estrategia real

→ señal esperada

Lo único falso es el sistema externo. El código propio se ejecuta realmente, incluyendo el esquema, las claves primarias, la idempotencia, el orden temporal y el cálculo de señales.

También existe un test espejo con una serie bajista y una señal esperada de -1. Esto evita que una estrategia defectuosa que siempre devolviese $+1$ superase el conjunto de pruebas.

11. Proceso de diseño, retos encontrados y decisiones de simplificación

11.1. Proceso de diseño del sistema

El sistema no se diseñó de una vez, sino de forma incremental y por fases, cada una construida sobre la anterior y validada antes de avanzar:

1. **Fase 0 — Repositorio y tooling:** estructura del proyecto, ruff, black y mypy desde el primer día, para que la calidad de código fuese una restricción permanente y no un repaso final.
2. **Fase 1 — Datos históricos y persistencia:** cliente REST con httpx y tenacity, esquema SQLite, entidades de dominio y validador de monotonicidad.
3. **Fase 2 — Ingesta en tiempo real:** WebSocket con asyncio, cola acotada y reconexión con backoff.
4. **Fase 3 — Estrategias:** tres estrategias bajo un contrato común impuesto por metaclase, con decorador de latencia.
5. **Fase 4 — Backend:** API FastAPI con endpoints de velas, señales y PnL.
6. **Fase 5 — Frontend:** dashboard Streamlit consumiendo exclusivamente la API.

El orden fue una decisión de diseño en sí misma. Empecé por la descarga histórica por REST y no por el WebSocket, aunque el enunciado destaca el tiempo real, porque el REST es determinista y fácil de depurar: permite fijar primero el esquema de almacenamiento, la deduplicación y la validación temporal con datos reproducibles. El WebSocket, que es la pieza más frágil (red, reconexiones, desorden), llegó cuando ya existía una base sólida sobre la que persistir. Del mismo modo, el backend se construyó antes que el frontend, porque el dashboard es un cliente de la API y no puede diseñarse antes que su contrato.

La arquitectura resultante es una separación en capas con una frontera estable:

```
Binance (REST / WebSocket)
        |
   Ingesta (asyncio, cola acotada, validación)
        |
   SQLite (WAL, claves primarias compuestas)
        |
   Backend FastAPI (consultas, señales, PnL)
        |
   Frontend Streamlit (solo HTTP)
```

El principio rector puede resumirse en una frase: *FastAPI es la frontera estable; SQLite es un detalle interno del backend*. El dashboard nunca abre `market.db` directamente: toda la lógica de negocio (consistencia, filtros, cálculo de PnL) vive en el backend, y el frontend es una vista. Esto es encapsulación a escala de sistema: si mañana SQLite se sustituyese por PostgreSQL, el dashboard no se enteraría, porque su contrato es con la API y no con el motor de datos.

Durante el desarrollo, el sistema se ejecutó como tres procesos independientes (ingestor, backend y frontend, cada uno en su terminal) que comparten la base de datos y el contrato HTTP, pero no memoria de proceso. Esta separación operacional fue un ensayo deliberado de la fase final: al dockerizar, cada proceso se convirtió de forma casi directa en un contenedor, sin rediseñar nada.

Por último, la documentación de decisiones se llevó en paralelo al código, no de forma retrospectiva. Cada fase cerraba con un checkpoint de preguntas respondidas sin mirar el código, y esas respuestas alimentaron directamente esta memoria. Así, las justificaciones que aparecen aquí reflejan el razonamiento en el momento de decidir, no una reconstrucción posterior.

11.2. Justificación de las principales elecciones

11.2.1. Arquitectura del sistema: capas con separación dominio/infraestructura (y por qué no MVP/MVC)

La elección de arquitectura era libre, así que el primer paso fue identificar dónde vive la complejidad del sistema, porque la arquitectura debe organizarse alrededor de ella. En este proyecto la complejidad no está en la interfaz de usuario, sino en el pipeline de datos: ingesta concurrente, idempotencia, orden temporal, backpressure y cálculo de PnL sin sesgo. Eso descartó los patrones de capa de presentación y orientó el diseño hacia una **arquitectura en capas (layered) con separación estricta entre dominio e infraestructura**, que admite dos lecturas complementarias:

- **Lectura lógica (capas):** dominio (`Trade`, `Candle`, `Position`, `MonotonicityValidator` — stdlib puro, sin dependencias externas) → aplicación (estrategias, cálculo de PnL) → infraestructura (SQLite, clientes REST y WebSocket de Binance, FastAPI) → presentación (Streamlit). La regla de dependencia se respeta en una sola dirección: las capas externas conocen a las internas, nunca al revés. El dominio no importa `httpx` ni `sqlite3`, y `parse_trade` actúa como aduana donde el formato propio de Binance se traduce a entidades del dominio, de modo que un cambio en el exchange no contamina el resto del sistema.
- **Lectura física (despliegue):** cliente-servidor en tres niveles (*3-tier*): presentación (dashboard) → lógica (API) → datos (SQLite), materializada en tres procesos independientes que solo comparten la base de datos y el contrato HTTP.

Por qué no MVP ni MVC. MVP y MVC son patrones de *capa de presentación*: resuelven cómo organizar una interfaz de usuario separando la vista del modelo mediante un intermediario (Presenter o Controller) que coordina eventos y actualizaciones. Nacieron para aplicaciones de escritorio y hoy viven en entornos como Android o frontends ricos, donde la complejidad está precisamente en la UI. Aplicarlos aquí habría sido estructurar todo el sistema alrededor de su parte más simple. Además, Streamlit ni siquiera encaja en el patrón: su modelo de ejecución re-ejecuta el script completo en cada interacción (un render loop declarativo), por lo que no existe un Presenter escuchando eventos que coordinar. En su lugar, el papel de la capa de presentación queda reducido deliberadamente a una vista sobre la API.

Relación con la arquitectura hexagonal. La hexagonal formal (puertos definidos como `Protocol`s, adaptadores intercambiables, dominio en el centro) es la versión ceremonializada de lo que este proyecto ya practica por disciplina: las fronteras existen (`parse_trade`, la capa de consultas, la API como frontera estable) sin los contratos formales. Que el cliente del exchange sea sustituible lo demuestra en la práctica el `FakeExchange` de los tests, que respeta el mismo contrato que el cliente real. Formalizar los puertos quedó conscientemente fuera de alcance (véase la subsección de simplificaciones): el trabajo conceptual está hecho y faltaría únicamente la formalización, que no cambiaría ninguna propiedad observable del sistema.

En resumen: la arquitectura se eligió en función de dónde vive la complejidad. MVP no aplicaba porque es un patrón de presentación y la complejidad de este sistema está en el pipeline; capas con separación dominio/infraestructura — hexagonal en espíritu — era la opción que protegía justo lo que había que proteger.

11.2.2. SQLite con WAL como almacenamiento

Elegí SQLite frente a PostgreSQL por tres razones. Primera, el sistema tiene un único proceso escritor (el ingestor), que es exactamente el caso para el que SQLite está pensado; un servidor de base de datos añadiría administración, credenciales y despliegue sin aportar ninguna garantía adicional a este escenario. Segunda, el modo WAL permite que el backend lea mientras el ingestor escribe, sin bloqueos entre lector y escritor, que es justo el patrón de este pipeline. Tercera, la base de datos es un único fichero, lo que simplifica la reproducibilidad y el montaje de volúmenes en Docker. La decisión no es irreversible: gracias a la frontera de la API, una migración futura a PostgreSQL quedaría encapsulada en el backend.

11.2.3. asyncio como modelo de concurrencia

El problema de la ingesta es I/O-bound: el proceso pasa casi todo el tiempo esperando mensajes de red, no calculando. Con el GIL, los hilos no aportarían paralelismo real de CPU, y el multiprocesamiento añadiría coste de comunicación entre procesos para un trabajo que no lo necesita. asyncio permite gestionar diez símbolos y la persistencia con un único hilo, cooperativamente, con un modelo productor–consumidor explícito mediante `asyncio.Queue`. Además, era la opción coherente con el resto de la pila: websockets y FastAPI son asíncronos de forma nativa.

11.2.4. FastAPI como backend

FastAPI se eligió por cuatro motivos: soporte asíncrono nativo (coherente con la ingesta), validación automática de entrada y salida mediante tipos y Pydantic, documentación OpenAPI/Swagger generada sin esfuerzo (que sirvió además como herramienta de prueba manual de los endpoints), y tipado estático compatible con mypy, alineado con el tooling del proyecto. Frente a Flask, ofrece estas garantías sin código adicional; frente a Django, evita cargar un framework completo para exponer media docena de endpoints de lectura.

11.2.5. Streamlit como frontend

La decisión más condicionada por el calendario. Evalué React y Plotly Dash y los descarté explícitamente: React exige una segunda pila tecnológica (JavaScript, build, estado) cuyo coste no se traduce en mejor nota en un proyecto cuyo centro de gravedad es el pipeline de datos, y Dash queda a medio camino con más fricción que Streamlit para el mismo resultado. Streamlit permitió construir en horas un dashboard funcional (velas con SMAs, señales, PnL, selector de símbolo y estrategia) permaneciendo en Python.

El coste asumido es que Streamlit funciona por *polling* y no por *push*: el dashboard consulta la API cada 2 segundos en modo tiempo real. Esta simplificación es defendible porque el requisito de tiempo real del enunciado se satisface donde importa: en la ingesta por WebSocket, que recibe y persiste cada trade en el momento en que ocurre. El dashboard es una vista sobre datos que ya son en tiempo real; llevar el push hasta el navegador duplicaría infraestructura para un beneficio visual de menos de dos segundos.

Al polling se le añadió una caché de dos velocidades con `@st.cache_data`: TTL de 2 segundos para datos vivos (velas, señales, PnL) y de 300 segundos para metadatos casi estáticos (lista de símbolos y estrategias). El TTL corto no es un número suelto: está alineado deliberadamente con el ciclo de refresco, porque un TTL mayor que el periodo de refresco produciría un tiempo real aparente que repinta datos viejos. La asimetría de riesgos guio la elección: un TTL demasiado largo produce datos falsos (fallo grave y silencioso); uno demasiado corto solo produce peticiones de más (fallo leve y medible). Ante la duda, se acorta.

11.2.6. Construcciones avanzadas de Python con propósito

Los requisitos de metaclase, decorador y dunder methods se integraron donde resolvían un problema real, no como adorno:

- **Metaclase** `StrategyMeta`: valida en tiempo de importación que toda estrategia implemente `generate_signals` con el contrato correcto y la registra automáticamente en `STRATEGY_REGISTRY`. Añadir una estrategia nueva no exige tocar código existente (principio abierto/cerrado), y un contrato incumplido falla al importar, no en producción.
- **Decorador** `measure_latency`: observabilidad transversal mediante `functools.wraps`, con soporte para funciones síncronas y asíncronas, sin contaminar la lógica de negocio con código de medición.
- **Dunder methods**: `__eq__` y `__hash__` por identidad de dominio `(symbol, trade_id)` sobre dataclasses con `frozen=True`, de modo que la noción de duplicado es la misma en memoria (sets, comparaciones) que en la base de datos (clave primaria compuesta).

11.3. Retos encontrados y cómo se resolvieron

11.3.1. Desconexión espontánea del WebSocket

Durante una ejecución real, Binance cerró la conexión con el error 1011 (*keepalive ping timeout*). No fue un fallo provocado en tests, sino un incidente real que validó el diseño de resiliencia: el ingestor aplicó la reconexión automática con backoff exponencial, jitter y reinicio del contador tras una conexión estable, y continuó ingiriendo sin intervención manual y sin duplicar datos gracias a la idempotencia de las inserciones. El jitter evita que múltiples clientes reintenten sincronizados tras una caída del servidor, y el reset tras éxito impide que un fallo puntual herede esperas largas de fallos antiguos.

11.3.2. Latencia aparente negativa y clock drift

Al medir `ingestion_time − event_time` esperaba obtener la latencia de red, pero obtuve valores negativos (una media de -1.502 ms en una sesión) y, además, el valor cambió de $+381$ ms a -290 ms entre sesiones distintas. Una latencia física no puede ser negativa: la explicación es que la resta entre timestamps de máquinas distintas contiene también el desfase entre relojes (*clock drift*) entre mi equipo Windows y los servidores de Binance. La resolución fue conceptual y de diseño: comparar timestamps de relojes no sincronizados no da una latencia absoluta con significado físico, así que la métrica se reinterpretó como monitor relativo (un salto brusco indica un problema) y se fijó la regla de usar siempre `event_time` para cálculos financieros y `ingestion_time` únicamente para observabilidad interna.

11.3.3. Símbolo inexistente en Binance spot

El universo inicial de activos incluía HYPEUSDT, que devolvía sistemáticamente `400 Bad Request`. Tras aislar la petición y verificarla contra la API, confirmé que el símbolo no existe en el mercado spot global de Binance (cotiza en otros ámbitos). Se sustituyó por otro activo líquido, y la lección incorporada fue no asumir la validez del universo de símbolos: los errores 4xx del exchange son errores de petición, no transitorios, y reintentarlos con backoff sería inútil, por lo que el tratamiento correcto es distinguirlos de los errores de red.

11.3.4. Duplicados por reconexión y solapamiento

Tanto las reconexiones del WebSocket como las descargas históricas solapadas pueden entregar el mismo trade varias veces. En lugar de deduplicar en la aplicación (mantener sets en memoria,

con estado que se pierde al reiniciar), la unicidad se delegó en el almacenamiento: clave primaria compuesta `(symbol, trade_id)` e inserciones con `INSERT OR IGNORE`. El resultado es idempotencia: reprocesar el mismo flujo produce el mismo estado final, lo que convierte la reconexión y el backfill en operaciones seguras por construcción.

11.3.5. Backpressure: el productor puede superar al consumidor

Con diez símbolos, en picos de mercado el WebSocket produce más rápido de lo que SQLite persiste. Una cola sin límite absorbería la diferencia consumiendo memoria sin cota hasta caer. La solución fue `asyncio.Queue(maxsize=1000)` con política de bloquear al productor cuando la cola se llena: el sistema deja de leer temporalmente del socket antes que crecer sin control o descartar datos en silencio. Es la opción que prioriza la integridad de los datos y hace el problema visible (la cola llena es observable) en lugar de ocultarlo.

11.3.6. Look-ahead bias en el cálculo de PnL

El error clásico de backtesting es ejecutar la señal en la misma vela con la que se calculó, usando implícitamente información futura. El PnL se calcula desplazando las señales una vela con `signals.shift(1)`: la señal generada al cierre de la vela t se ejecuta en $t{+}1$. El dashboard pinta la señal en t (cuándo se decidió) y el PnL la aplica en $t{+}1$ (cuándo pudo ejecutarse); no es una inconsistencia, sino la distinción entre decisión y ejecución. Los resultados obtenidos (por ejemplo, momentum con $-3{,}97\,\%$ total pero $+6{,}86\,\%$ en julio) son coherentes con el perfil esperado de una estrategia seguidora de tendencia, lo que dio confianza cualitativa en el cálculo.

11.3.7. Validar el desorden temporal sin poder provocarlo

En 14.200 trades reales observé cero violaciones de monotonicidad, así que no podía apoyarme en la red para probar el validador. La solución fue testear con secuencias sintéticas deterministas donde el número y la magnitud de las violaciones se conocen de antemano, y aplicar la misma filosofía al resto del sistema: un `FakeExchange` que respeta el contrato del cliente real permite ejecutar el pipeline completo (SQLite real, consultas reales, estrategias reales) con la única pieza falsa en el sistema externo.

11.4. Decisiones de simplificación adoptadas

Todas las simplificaciones se tomaron de forma explícita, con su justificación registrada en el momento de decidir, y bajo un criterio común: proteger el núcleo del sistema y su comprensión profunda frente a la acumulación de piezas periféricas. Un proyecto con menos componentes, todos defendibles, vale más que uno con más componentes a medio entender.

- **Streamlit en lugar de React o Plotly Dash.** El frontend es una vista sobre la API; su sofisticación tecnológica no añade valor al pipeline, que es el objeto real de evaluación. React habría consumido un tiempo desproporcionado en una segunda pila tecnológica.
- **Sin arquitectura hexagonal formal con Protocols.** El beneficio esencial de la hexagonal (aislar el dominio de la infraestructura tras fronteras estables) ya está capturado de forma pragmática: el dashboard solo conoce la API, las estrategias solo conocen el contrato de `generate_signals`, y el `FakeExchange` demuestra que el cliente del exchange es sustituible. Formalizarlo con puertos y adaptadores explícitos habría añadido ceremonia sin cambiar ninguna propiedad del sistema.
- **Sin Redis.** Una caché externa se justifica cuando varios procesos deben compartir caché o cuando esta debe sobrevivir a reinicios. Aquí hay un único proceso de frontend, y

`@st.cache_data` con doble TTL resuelve el mismo problema dentro del proceso, sin desplegar ni operar otro servicio.

- **Sin Terraform.** Infraestructura como código presupone infraestructura en la nube que aprovisionar; este proyecto se despliega con docker-compose en una máquina. Terraform habría sido tecnología sin problema que resolver.
- **Tests unitarios y de pipeline, sin suite e2e completa.** Los tests cubren lo que puede romperse con lógica propia: estrategias con resultados calculados a mano, validador de monotonicidad con secuencias deterministas, e integración del pipeline con `FakeExchange`. Una suite end-to-end contra Binance real sería no determinista (dependiente de red y de mercado) y lenta, comprometiendo justo las propiedades que hacen útil un test.
- **SQLite con `synchronous=NORMAL` en lugar de `FULL`.** Se aceptó conscientemente el riesgo de perder el último commit ante un corte brusco de alimentación a cambio de mejor latencia de escritura, un intercambio razonable para datos de mercado que pueden reconstruirse desde la fuente histórica.
- **Polling en el dashboard en lugar de push hasta el navegador.** Ya justificado: el tiempo real vive en la ingesta; la vista tolera 2 segundos de retardo.
- **Detección de desorden temporal en lugar de reordenación con watermark.** El validador detecta y cuantifica retrocesos; la reconstrucción con buffer y marca de agua está diseñada (sección 1) pero no implementada, porque en 14.200 trades reales no se observó ni una violación: implementar la corrección habría sido resolver un problema que los datos aún no presentan, manteniendo no obstante la instrumentación para detectarlo cuando aparezca.

En conjunto, el proceso siguió una disciplina constante: decidir explícitamente, registrar el porqué, validar cada fase antes de avanzar y recortar en la periferia para no comprometer el núcleo. Las simplificaciones no son carencias ocultas, sino intercambios documentados cuyo camino de evolución (PostgreSQL tras la frontera de la API, buffer con watermark sobre el validador existente, contenedores ya alineados con los procesos) queda trazado en las secciones anteriores.